BENCHMARKING NETWORK TRAFFIC FORECASTING

Ondřej Skácel

Bachelor's thesis
Faculty of Information Technology
Czech Technical University in Prague
Katedra aplikované matematiky
Study program: Informatika
Specialisation: Umělá Inteligence
Supervisor: Ing. Josef Koumar
May 15, 2026

FACULTY
OF INFORMATION
TECHNOLOGY
CTU IN PRAGUE

Assignment of bachelor's thesis

Title:	Benchmarking Network Traffic Forecasting
Student:	Ondřej Skácel
Supervisor:	Ing. Josef Koumar
Study program:	Informatics
Branch / specialization:	Artificial Intelligence 2021
Department:	Department of Applied Mathematics
Validity:	until the end of summer semester 2026/2027

Instructions

Study computer network monitoring and time series analysis, with a specific focus on time series forecasting [1-2]. Explore the CESNET-TimeSeries24 dataset [3], which contains volumetric information about IP addresses and networks from a high-speed ISP network.

Create comprehensive, smaller time series benchmarks that share similar characteristics (e.g., seasonality, communication occurring only during working hours, the presence of data drifts, etc.). Select several state-of-the-art (SOTA) time series forecasting models (for example, Prophet [4], TimesFM [5], and Chronos [6]).

Develop a prototype system for the automated evaluation of these SOTA models using the created benchmarks. Finally, compare and analyze the performance results of the selected models.

[1] Lim, Bryan, and Stefan Zohren. "Time-series forecasting with deep learning: a survey." Philosophical transactions of the royal society a: mathematical, physical and engineering sciences 379.2194 (2021).
[2] Benidis, Konstantinos, et al. "Deep learning for time series forecasting: Tutorial and literature survey." ACM Computing Surveys 55.6 (2022): 1-36.
[3] Koumar, Josef, et al. "CESNET-TimeSeries24: Time Series Dataset for Network Traffic Anomaly Detection and Forecasting." Scientific Data 12.1 (2025): 338.
[4] https://github.com/facebook/prophet

Electronically approved by Ing. Magda Friedjungová, Ph.D. on 10 January 2026 in Prague.

[5] https://github.com/google-research/timesfm
[6] https://github.com/amazon-science/chronos-forecasting

Czech Technical University in Prague
Faculty of Information Technology
© 2026 Ondřej Skácel. All rights reserved.
This thesis is school work as defined by the Copyright Act of the Czech Republic. It has been submitted to the Czech Technical University in Prague, Faculty of Information Technology. The thesis is protected by the Copyright Act and its use, with the exception of free statutory licenses and beyond the scope of the authorisations specified in the Declaration, requires the consent of the author.

Citation of this thesis: Skácel Ondřej. *Benchmarking Network Traffic Forecasting.* Bachelor's thesis. Czech Technical University in Prague, Faculty of Information Technology, 2026.

I would like to thank my supervisor, Ing. Josef Koumar, for his expert guidance, valuable advice and willingness throughout the development of this thesis. My heartfelt thanks also go to my family, whose unwavering support carried me through my studies. I further thank the CESNET association for providing computational resources through the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth and Sports of the Czech Republic. Lastly, I thank my friends for making the years at university truly memorable.

Declaration

I hereby declare that the presented thesis is my own work and that I have cited all sources of information in accordance with the Guideline for adhering to ethical principles when elaborating an academic final thesis.

I acknowledge that my thesis is subject to the rights and obligations stipulated by the Act No. 121/2000 Coll., the Copyright Act, as amended. In accordance with Section 2373(2) of Act No. 89/2012 Coll., the Civil Code, as amended, I hereby grant a non-exclusive authorization (licence) to utilize this thesis, including all computer programs that are part of it or attached to it and all documentation thereof (hereinafter collectively referred to as the "Work"), to any and all persons who wish to use the Work. Such persons are entitled to use the Work in any manner that does not diminish the value of the Work and for any purpose (including use for profit). This authorisation is unlimited in time, territory and quantity.

In Prague on May 15, 2026

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ
FACULTY OF INFORMATION TECHNOLOGY
Thákurova 9
160 00 Praha 6

DECLARATION

I, the undersigned

Student's surname, given name(s):	Skácel Ondřej
Personal number:	525132
Programme name:	Informatics

hereby declare that the Bachelor's Thesis titled

Benchmarking Network Traffic Forecasting

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 13.05.2026

Ondřej Skácel
.....
student's signature

Abstract

Network traffic forecasting supports capacity planning, anomaly detection, and network security, yet network traffic is absent from existing time series benchmarks. Existing evaluations aggregate diverse time series into a single ranking, which masks model failures on specific data characteristics.

This thesis presents five diagnostic benchmarks from the CESNET-TimeSeries24 dataset, each isolating one structural property (seasonality, data drift, periodic intermittent activity, workstation-driven schedules, or the absence of temporal structure) through predefined selection criteria. The resulting suite comprises 532 time series across three aggregation levels and two temporal resolutions. Ten forecasting models from statistical, deep learning, and zero-shot foundation model families are compared using a framework implemented as part of this work.

Foundation models achieve the best overall ranks, reducing forecasting error by up to 36 % over the baseline. However, rankings shift across benchmarks: recurrent models surpass all foundation models on workstation activity patterns. These benchmark-dependent differences would be invisible under a single aggregate ranking.

Keywords network traffic forecasting, benchmarking, time series forecasting, deep learning, zero-shot forecasting, diagnostic benchmarks, foundation models, CESNET-TimeSeries24, time series analysis

Abstrakt

Předpovídání síťového provozu je důležité pro plánování kapacity, detekci anomálií a zabezpečení sítí, přesto síťový provoz v existujících benchmarcích pro časové řady zcela chybí. Stávající hodnocení slučují různorodé časové řady do jednoho žebříčku, čímž zakrývají selhání modelů na datech s konkrétními vlastnostmi.

V rámci této práce bylo vytvořeno pět diagnostických benchmarků z datasetu CESNET-TimeSeries24, z nichž každý izoluje jednu strukturální vlastnost (sezónnost, datový drift, periodickou přerušovanou aktivitu, režim pracovních stanic nebo absenci časové struktury) pomocí předem definovaných selekčních kritérií. Výsledná sada obsahuje 532 časových řad na třech úrovních agregace a ve dvou časových rozlišeních. Deset predikčních modelů z rodin statistických modelů, hlubokého učení a zero-shot foundation modelů je porovnáno pomocí frameworku implementovaného v rámci této práce.

Foundation modely dosahují nejlepších celkových pořadí a snižují chybu předpovědi až o 36 % oproti referenční metodě. Pořadí se ale liší napříč benchmarky: rekurentní modely překonávají všechny foundation modely na vzorcích aktivity pracovních stanic. Tyto rozdíly závislé na typu benchmarku by byly skryty pod jedním souhrnným žebříčkem.

Klíčová slova předpovídání síťového provozu, benchmarking, předpovídání časových řad, hluboké učení, zero-shot předpovídání, diagnostické benchmarky, predikce síťového provozu, CESNET-TimeSeries24, analýza časových řad

Contents

Introduction **1**

1 Theoretical Background **3**
1.1 Computer Network Monitoring 3
1.2 Time Series Analysis 4
1.3 Time Series Forecasting 9
1.4 Evaluation Measures for Forecasting 11

2 Existing Related Works **13**
2.1 Network Traffic Forecasting 13
2.2 Time Series Benchmarking 14
2.3 Foundation Models for Time Series 15

3 CESNET-TimeSeries24 Dataset **17**
3.1 Data Collection and Access 17
3.2 Scope, Time Range, and Resolution 18
3.3 Aggregation Levels 18
3.4 Feature Set 19
3.5 Properties of `n_bytes` 19
3.5.1 Sparsity 19
3.5.2 Distribution 20

4 Benchmarks **22**
4.1 Overview 22
4.2 Common Methodology 23
4.3 Seasonality Benchmark 24
4.4 Data Drift Benchmark 25
4.5 Periodic Spaces Benchmark 27
4.6 Workstation Activity Benchmark 29
4.7 No Temporal Structure Benchmark 31
4.8 Summary 32

5 Evaluation Methodology **34**
5.1 Selected Forecasting Models 34
5.2 Data Preparation and Evaluation Protocol 36
5.3 Per-Series Tuning 38
5.4 Evaluation Measures 39

5.5 Result Aggregation and Ranking 40

6 Framework **43**
6.1 Benchmark Data Management 44
6.2 Model Abstraction 44
6.3 Evaluation Engine 46
6.4 Command-Line Interface 47
6.5 Result Analysis and High-Performance Computing (HPC) Deployment 47

7 Experiments and Results **48**
7.1 Overall Model Ranking 48
7.2 Statistical Significance 50
7.3 Pairwise Comparisons 51
7.4 Sensitivity to Configuration 53
7.5 Summary 54

8 Discussion **56**

9 Conclusion **58**

A `n_bytes` Properties at 10-minute Resolution **59**

B Selected Benchmark Series **60**

C Threshold Sensitivity Analysis **64**
C.1 Seasonality 64
C.2 Data Drift 65
C.3 Periodic Spaces 68
C.4 Workstation Activity 70
C.5 No Temporal Structure 72

D Friedman Test Statistics and Critical Difference Diagrams **74**

E Inference Speed **81**

Bibliography **82**

Attachment contents **89**

List of Figures

1.1 Architecture of CESNET-TimeSeries24 dataset collection [17]. . 4
1.2 Forecasting task setup with context window and forecast horizon. 5
1.3 Samples of autocorrelation functions. 6
1.4 Patterns of changes over time (outlier is not concept drift) [28]. 8

3.1 Raw vs. log-transformed `n_bytes` distribution. 21

4.1 Seasonal and Trend decomposition using Loess (STL) decomposition for Institution 15. 25
4.2 Drift detection for Institution 209. 27
4.3 Periodic spaces detection for IP 722 293. 29
4.4 Workstation activity pattern for IP 21 943. 30
4.5 Autocorrelation Function (ACF) verification for Subnet 89 (No Temporal Structure). 32

5.1 Overview of the data preparation and evaluation protocol. . . . 37
5.2 Non-overlapping sliding-window evaluation scheme [22]. 38

7.1 Mean MASE by aggregation level. 50
7.2 CD diagram: Data Drift, h/744/168. 51
7.3 CD diagram: Workstation Activity, m/1008/144. 51
7.4 Pairwise win rate heatmap. 52
7.5 Pairwise skill score heatmap. 52

C.1 Seasonality: score distributions. 65
C.2 Seasonality: pool-size sweep. 65
C.3 Data Drift deviation: score distributions. 66
C.4 Data Drift deviation: pool-size sweep. 67
C.5 Data Drift persistence: pool-size sweep. 67
C.6 Periodic Spaces: score distributions. 69
C.7 Periodic Spaces: pool-size sweep. 69
C.8 Workstation Activity: score distributions. 71
C.9 Workstation Activity: pool-size sweep. 72
C.10 No Temporal Structure: score distributions. 73
C.11 No Temporal Structure: pool-size sweep. 73

D.1 CD diagrams: Data Drift across all four configurations. 76

D.2 CD diagrams: Seasonality across all four configurations. 77
D.3 CD diagrams: Periodic Spaces across all four configurations. . . 78
D.4 CD diagrams: Workstation Activity across all four configurations. 79
D.5 CD diagrams: No Temporal Structure across all four configurations. 80

List of Tables

2.1 Comparison of representative time series forecasting benchmarks. 15

3.1 CESNET-TimeSeries24 aggregation levels. 18
3.2 Per-datapoint metrics in CESNET-TimeSeries24. 19
3.3 Percentage of non-zero datapoints per aggregation level. 20
3.4 Per-series distribution statistics of `n_bytes`. 20

4.1 Seasonality: candidate counts (hourly). 25
4.2 Data Drift: candidate counts (hourly). 26
4.3 Periodic Spaces: candidate counts (hourly). 28
4.4 Workstation Activity: candidate counts (hourly). 30
4.5 No Temporal Structure: candidate counts (hourly). 31
4.6 Summary of the five diagnostic benchmarks. 33

5.1 Evaluation configurations. 36

6.1 Libraries and model sizes for each model wrapper. 46

7.1 Average MASE rank per benchmark. 48
7.2 Skill scores relative to Seasonal Naïve per benchmark. 49
7.3 Average rank per configuration. 53
7.4 Skill score per configuration. 53

A.1 Percentage of non-zero datapoints per aggregation level (10-minute). 59
A.2 Per-series distribution statistics of `n_bytes` (10-minute). 59

B.1 Seasonality: candidate counts (10-minute). 60
B.2 Data Drift: candidate counts (10-minute). 60
B.3 Periodic Spaces: candidate counts (10-minute). 61
B.4 Workstation Activity: candidate counts (10-minute). 61
B.5 No Temporal Structure: candidate counts (10-minute). 61
B.6 Selected series for 10-minute aggregation. 62

B.7 Selected series for 1-hour aggregation. 63

C.1 Data Drift: calendar-week coverage. 68
C.2 Periodic Spaces: activity-band comparison. 70
C.3 Workstation Activity: joint-binding analysis. 71

D.1 Friedman test statistics. 75

E.1 Wall-clock time per series. 81

List of Code Listings

1 Model interface. 45

List of Abbreviations

ACF Autocorrelation Function

ARIMA Autoregressive Integrated Moving Average

CD Critical Difference

CSV Comma-Separated Values

GRU Gated Recurrent Unit

GRU-FCN Gated Recurrent Unit - Fully Convolutional Network

HPC High-Performance Computing

IETF Internet Engineering Task Force

IP Internet Protocol

IPFIX IP Flow Information Export

ISP Internet Service Provider

LSTM Long Short-Term Memory

MA Moving Average

MAE Mean Absolute Error

MASE Mean Absolute Scaled Error

RMSE Root Mean Squared Error

RNN Recurrent Neural Network

SARIMA Seasonal Autoregressive Integrated Moving Average

SMAPE Symmetric Mean Absolute Percentage Error

STL Seasonal and Trend decomposition using Loess

TTL Time To Live

TTM Tiny Time Mixers

WHR Working Hours Ratio

Introduction

Global internet traffic continues to grow with the proliferation of connected devices, cloud computing, and streaming [1], making accurate traffic forecasting essential for capacity planning, anomaly detection, and resource allocation [2].

The underlying time series methods have progressed from classical statistical models such as Autoregressive Integrated Moving Average (ARIMA) [3] through deep learning architectures [4, 5] to pre-trained foundation models [6, 7]. Models such as TimesFM [8], Chronos-2 [9], and Moirai [10] are trained on large heterogeneous corpora and applied zero-shot to unseen domains. Whether this generalization extends to network traffic—characterized by strong daily and weekly periodicity, burstiness and heavy-tailed volume distributions, and distribution drift—remains an open question [11].

Existing evaluation benchmarks are ill-suited to answer it. Mixed-domain collections such as the M4 competition [12], the Monash Archive [13], and GIFT-Eval [14] aggregate structurally diverse series into a single ranking, where accuracy averaged over heterogeneous data masks systematic failures on specific data characteristics [15]. Domain-specific evaluations like M5 [16] yield more actionable conclusions, yet no comparable benchmark suite exists for network traffic.

This thesis addresses the gap by constructing five property-specific diagnostic benchmarks from the CESNET-TimeSeries24 dataset [17], a large-scale collection of network traffic time series at multiple aggregation intervals from the Czech national research and education network. Each benchmark isolates a single structural characteristic—seasonality, data drift, periodic intermittent activity, workstation-driven schedules, or the absence of temporal structure—so that performance differences can be attributed to the property under test. Forecasting models spanning statistical, deep learning, and foundation model categories are then evaluated against these benchmarks through a unified sliding-window framework implemented as part of this work.

Goals

The goals of this thesis are to design and create property-specific diagnostic benchmarks from the CESNET-TimeSeries24 dataset, each isolating a distinct network traffic pattern. Based on a survey of current forecasting approaches, state-of-the-art models spanning statistical, deep learning, and foundation model categories are selected and configured. An evaluation framework is then implemented to automatically assess these models against the created benchmarks. Finally, the performance of the selected models is compared and analyzed.

Chapter 1
Theoretical Background

This chapter introduces the concepts required to follow the practical chapters. Section 1.1 outlines how network traffic data is collected and organized, providing the context for the dataset described in Chapter 3. Section 1.2 reviews time series properties utilized by the benchmark selection pipelines in Chapter 4. Section 1.3 surveys the forecasting model families evaluated in this work, and Section 1.4 defines the error metrics used throughout the evaluation.

1.1 Computer Network Monitoring

Network monitoring is the practice of continuously observing traffic traversing a computer network to support capacity planning, performance management, anomaly detection, and security enforcement [18]. Two broad approaches exist. Packet-based analysis, commonly referred to as deep packet inspection, examines the full content of every packet, including its payload. While this yields the richest information, the computational cost grows linearly with link speed, making it impractical on backbone links that carry hundreds of gigabits per second [18]. Flow-based monitoring addresses this scalability limitation by aggregating packets into flows and exporting only summary statistics without using payload data. The resulting reduction in data volume enables monitoring at line rate on high-speed networks, which is why flow-based monitoring has become the prevalent approach in production Internet Service Provider (ISP) networks [18]. The CESNET-TimeSeries24 dataset used in this work (Chapter 3) is derived from flow-based monitoring infrastructure.

Flow-Based Monitoring

A network flow is defined as a set of Internet Protocol (IP) packets passing an observation point during a certain time interval that share a common set of properties [19]. In practice, packets are grouped into flows by a 5-tuple key:

source and destination IP addresses, source and destination ports, and transport protocol. The monitoring device (exporter) maintains per-flow counters and exports a flow record once the flow terminates or an active/idle timeout expires. Beyond the key fields, each record carries aggregate counters such as packet and byte counts, Time To Live (TTL), and timestamps, but no payload [18].

The first widely adopted flow export protocol was NetFlow, developed by Cisco. Its version 9 introduced a template-based record format that decoupled the export structure from a fixed field layout [20]. The Internet Engineering Task Force (IETF) subsequently standardized this template-based design as the IP Flow Information Export (IPFIX) protocol [19], which is the format used by the CESNET3 monitoring infrastructure. There, the open-source exporter `ipfixprobe` [21] runs on monitoring probes placed before edge routers, creating bidirectional flow records with a 5-minute active timeout and a 65-second idle timeout [17].

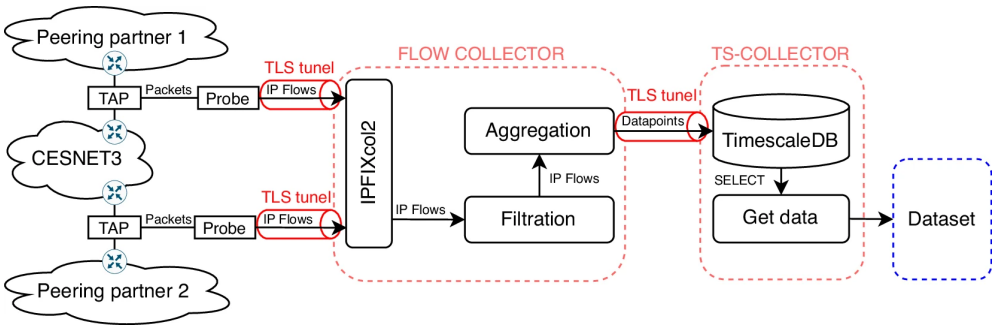


Figure 1.1 Architecture of CESNET-TimeSeries24 dataset collection [17].

Flow records can be aggregated at different spatial scopes to produce time series at varying granularities. In the CESNET-TimeSeries24 dataset, three hierarchical levels are defined: individual IP addresses, institution subnets, and whole institutions [17]. The choice of aggregation level has a direct effect on the statistical properties of the resulting series. At the institution level, traffic from hundreds or thousands of endpoints is summed, smoothing out short-lived bursts and emphasizing periodic patterns such as daily and weekly cycles. At the IP level, individual device behavior is preserved, producing a heterogeneous population that ranges from always-on servers to sporadically active workstations [22]. This trade-off between smoothness and behavioral resolution is further discussed in Section 3.3.

1.2 Time Series Analysis

A *time series* is an ordered sequence of observations indexed by time. In this work, all series are discrete and uniformly spaced: $\{y_t\}_{t=1}^{T}$ denotes T

observations recorded at a fixed sampling interval (one hour in the primary resolution).

A forecasting task is defined by two parameters. The *context window* (or lookback) of length L provides historical observations $y_{t-L+1}, \ldots, y_t$ as input. The *forecast horizon* H is the number of future steps the model must predict: $\hat{y}_{t+1}, \ldots, \hat{y}_{t+H}$. Increasing H generally increases prediction difficulty because the model must extrapolate further beyond the observed data [23]. Figure 1.2 illustrates this setup.

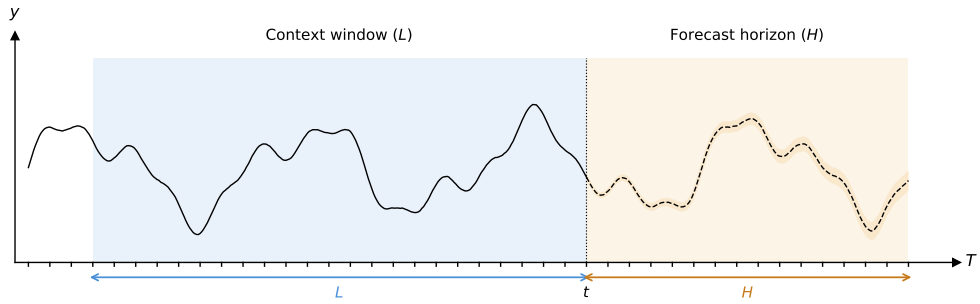


Figure 1.2 Forecasting task setup. The model receives L historical observations (context window) and produces H future predictions (forecast horizon).

The *autocovariance function* of a stationary process $\{Y_t\}$ with mean μ measures the linear dependence between observations separated by k time steps [3]:

$$\gamma(k) = \mathrm{Cov}(Y_t, Y_{t+k}) = \mathrm{E}[(Y_t - \mu)(Y_{t+k} - \mu)]\,. \tag{1.1}$$

At lag zero, $\gamma(0) = \mathrm{Var}(Y_t)$.

Stationarity

A stochastic process is *strictly stationary* if its properties are unaffected by a change of time origin. Formally, the joint probability distribution of m observations $Y_{t_1}, Y_{t_2}, \ldots, Y_{t_m}$ must equal that of $Y_{t_1+k}, Y_{t_2+k}, \ldots, Y_{t_m+k}$ for any times $t_1, \ldots, t_m$ and any integer shift k. Setting $m = 1$ shows that the marginal distribution is identical at every time point, so both the mean $\mathrm{E}[Y_t] = \mu$ and the variance $\mathrm{Var}(Y_t) = \sigma^2$ are constant. Setting $m = 2$ shows that the joint distribution of (Y_t, Y_{t+k}) depends only on the lag k, not on t, so the autocovariance $\gamma(k) = \mathrm{Cov}(Y_t, Y_{t+k})$ is likewise time-invariant. When this assumption is violated, for instance by a trend or a level shift, models fitted on past data may systematically mispredict future values [3].

Autocorrelation Function

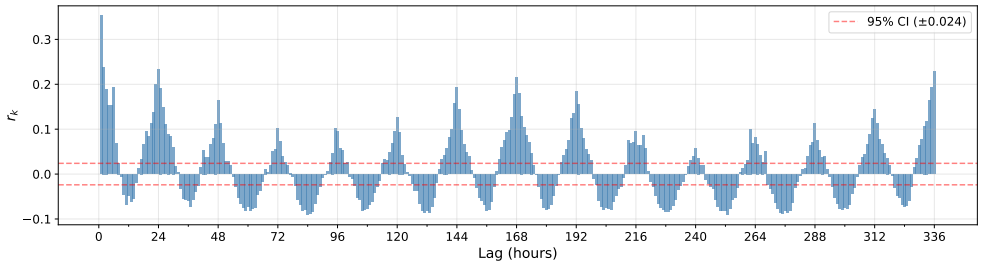
The ACF quantifies the linear dependence between observations separated by k time steps. For a stationary process with autocovariance $\gamma(k)$, the autocorrelation at lag k is defined as [3]:

$$\rho(k) = \frac{\gamma(k)}{\gamma(0)} = \frac{\mathrm{Cov}(Y_t, Y_{t+k})}{\mathrm{Var}(Y_t)}. \tag{1.2}$$

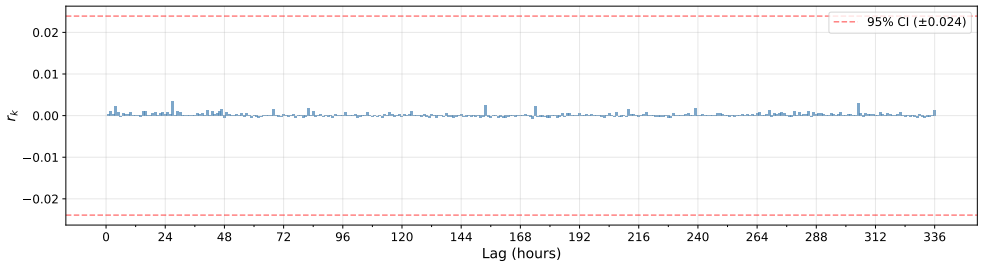
By construction $\rho(0) = 1$ and $|\rho(k)| \leq 1$. In practice, the sample ACF is estimated from n observations as:

$$r_k = \frac{\sum_{t=1}^{n-k}(y_t - \bar{y})(y_{t+k} - \bar{y})}{\sum_{t=1}^{n}(y_t - \bar{y})^2}.$$

Under the null hypothesis that the underlying process is white noise, Bartlett [24] showed that r_k is approximately normally distributed with variance $1/n$, yielding the standard 95 % confidence band $\pm 1.96/\sqrt{n}$. Values of r_k exceeding this band indicate statistically significant linear dependence at lag k. Figure 1.3 contrasts two correlograms computed from the CESNET-TimeSeries24 dataset.



(a) Strongly seasonal institution-level series with prominent daily (lag 24) and weekly (lag 168) peaks.



(b) Subnet-level series with no temporal structure. All values of r_k remain within the 95 % confidence band.

Figure 1.3 Sample autocorrelation functions of two CESNET-TimeSeries24 series over 336 lags (two weeks). The dashed lines mark the Bartlett 95 % confidence band.

Time Series Decomposition

Decomposition separates an observed series into interpretable components. The additive model assumes:

$$y_t = T_t + S_t + R_t,$$

where T_t is the trend-cycle, S_t is the seasonal component, and R_t is the remainder. A multiplicative variant ($y_t = T_t \cdot S_t \cdot R_t$) is appropriate when seasonal amplitude scales with the level. Applying a log-transformation before decomposition converts multiplicative structure to additive form.

STL is an iterative decomposition procedure that uses locally weighted polynomial regression (LOESS) to estimate the trend and seasonal components. It operates through two nested loops. The *inner loop* alternates between detrending and cycle-subseries smoothing: first T_t is subtracted, then each of the m subseries (one per seasonal position) is smoothed independently with a LOESS window of width n_s, producing the seasonal estimate S_t. A low-pass filter removes residual trend leakage from S_t. After deseasonalizing, T_t is re-estimated with a separate LOESS window n_t. The *outer loop* assigns robustness weights based on the magnitude of R_t, downweighting observations with large remainders in subsequent iterations to reduce the influence of outliers [25].

Compared to classical decomposition, which uses fixed moving averages and assumes a time-invariant seasonal pattern, STL allows both the seasonal shape and trend to evolve over time. The degree of temporal variation in S_t is controlled by the seasonal window parameter n_s: larger values enforce a more stable seasonal shape, while smaller values allow the pattern to adapt to local changes [25].

Seasonality and Periodicity

A time series exhibits *seasonality* when a pattern repeats at a fixed, known period m. In network traffic, the two dominant periods are daily ($m = 24$) and weekly ($m = 168$), reflecting working-hour activity and the weekday–weekend contrast [26]. Seasonality has a fixed period length, whereas periodicity (a cyclical pattern) has no fixed period [27].

To quantify how strongly a series is governed by its seasonal component, the seasonal strength metric introduced by Wang et al. [27] is used:

$$F_S = \max\left(0,\ 1 - \frac{\mathrm{Var}(R)}{\mathrm{Var}(S+R)}\right), \tag{1.3}$$

where S and R are the seasonal and remainder components from the STL decomposition described in Section 1.2. $F_S \in [0, 1]$: values near 1 indicate that variance is dominated by the seasonal component rather than noise, while values near 0 indicate negligible seasonality.

Trend and Data Drift

A *trend* is a long-term systematic change in the level of a time series. Trends violate the constant-mean assumption of stationarity and, when unaccounted for, cause forecasting models to underestimate or overestimate future values.

When the statistical properties of a time series change over time in a sustained manner, the phenomenon is called *concept drift* [28]. Formally, drift occurs between time points t_0 and t_1 when $p_{t_0}(X, y) \neq p_{t_1}(X, y)$, that is, the joint distribution of features and target values changes. Gama et al. [28] distinguish four temporal patterns of drift:

- **Sudden** — the distribution changes abruptly at a single point.
- **Gradual** — during a transition period, observations alternate between old and new distributions with increasing probability of the new one.
- **Incremental** — the distribution shifts through many small consecutive steps.
- **Recurring** — a previously observed distribution reappears (e.g., seasonal load patterns).

In network traffic, drift can arise from changes in network configurations, user behaviors, or network conditions [29].

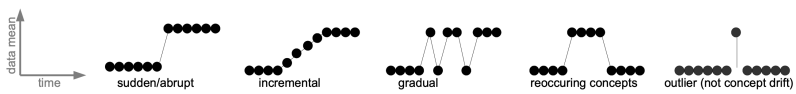


Figure 1.4 Patterns of changes over time (outlier is not concept drift) [28].

Sparsity

A time series is *sparse* (or intermittent) when a substantial fraction of its observations are zero. Sparse series pose problems for the analysis methods discussed earlier in this section. When most observations are zero, the sample ACF reflects the on/off pattern of activity rather than the temporal dynamics within active periods. Similarly, STL decomposition becomes uninformative on sparse data. Each cycle subseries (e.g., all Monday-3 AM observations) is smoothed independently by LOESS [25]. When most values in a subseries are zero, the local polynomial fit is pulled toward zero, producing $S_t \approx 0$ regardless of whether the non-zero values contain seasonal structure. The prevalence of sparsity in the dataset used in this work is quantified in Section 3.5.1.

1.3 Time Series Forecasting

Using the context window and forecast horizon defined in Section 1.2, this section introduces the three forecasting paradigms evaluated in this work. All models produce point forecasts. Probabilistic models that output full predictive distributions are reduced to point forecasts by taking the median. The specific models selected for evaluation and their configurations are detailed in Section 5.1.

Statistical Models

ARIMA models express each observation as a linear combination of its own past values (autoregressive terms) and past forecast errors (moving average terms), with differencing applied to achieve stationarity [3, 23]. An ARIMA(p, d, q) model is specified by three *orders*: p autoregressive lags, d differences, and q moving average lags. When the data exhibits seasonal patterns with period m, the model is extended to Seasonal Autoregressive Integrated Moving Average (SARIMA) by adding a seasonal component with its own orders $(P, D, Q)_m$ that operates on observations separated by m time steps. Building an ARIMA model is a two-level problem: first the orders that define the model structure must be selected, then the numerical coefficients (the autoregressive and moving average parameters) within that structure are estimated by maximum likelihood [3]. The classical Box–Jenkins methodology selects orders through manual inspection of the ACF and partial autocorrelation patterns, followed by iterative diagnostic checking [3]. Modern implementations automate this step by searching over candidate orders and selecting the combination that minimizes an information criterion such as AIC [23]. SARIMA is the dominant statistical baseline in network traffic forecasting [2].

Prophet [30] takes a decomposition-based approach, expressing the forecast as:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t,$$

where $g(t)$ is a piecewise-linear or logistic trend, $s(t)$ is a Fourier-based seasonal component that can capture multiple periodicities simultaneously, $h(t)$ accounts for the effect of holidays or known events, and ϵ_t is the error term. Unlike ARIMA, where the analyst must determine the model structure (order selection and differencing), Prophet uses a fixed decomposition structure. The analyst instead configures interpretable inputs such as holiday calendars, trend changepoint locations, and seasonal period counts. Components are estimated jointly using a Bayesian framework [30].

Deep Learning Models

Recurrent Neural Network (RNN)s process sequential data by maintaining a hidden state $\mathbf{h}_t$ that is updated at each time step as a function of the current input x_t and the previous state $\mathbf{h}_{t-1}$ [4]. This recurrence enables the network to capture temporal dependencies of arbitrary length in principle. In practice, standard RNNs suffer from vanishing gradients: during backpropagation through many time steps, gradient magnitudes decay exponentially, preventing the network from learning long-range dependencies [31].

Long Short-Term Memory (LSTM) [31] addresses this limitation by introducing a gated memory cell. Three gates — input, forget, and output — control what information is written to, retained in, and read from the cell state $\mathbf{c}_t$. The cell state provides a direct path through which gradients can flow across many time steps with minimal decay, enabling the learning of long-range dependencies.

Gated Recurrent Unit (GRU) [32] simplifies the LSTM architecture by merging the cell state and hidden state into a single vector and reducing the gating mechanism to two gates: a reset gate and an update gate. The update gate interpolates between the previous hidden state and a candidate activation, serving the combined role of the LSTM forget and input gates. This reduction in parameters lowers computational cost while achieving comparable performance on many sequence modeling tasks [5].

Gated Recurrent Unit - Fully Convolutional Network (GRU-FCN) [33] is a hybrid architecture that combines a recurrent branch with a convolutional branch. The convolutional branch applies one-dimensional convolutions along the time axis, extracting local patterns independently of sequential order. The outputs of both branches are concatenated before the final prediction layer, allowing the model to use both the long-range dependencies captured by the recurrent component and the local temporal features captured by the convolutions.

When applied to forecasting, these architectures are typically trained individually per series: a separate model is fitted to each time series using that series' training data [5]. This per-series paradigm avoids the need to learn cross-series patterns but incurs high computational cost when many series are involved and prevents transferring knowledge between related series.

Pre-trained Foundation Models

Foundation models for time series take the opposite approach to per-series training: a single large model is pre-trained on a diverse corpus of time series from multiple domains and subsequently applied to unseen series without any weight updates, a setting called *zero-shot* forecasting [6].

The model learns temporal patterns such as trends, seasonalities, level shifts, and intermittent activity from millions of heterogeneous training series.

At inference time, it conditions on a context window from a previously unseen series and generates forecasts without domain-specific adaptation. The underlying hypothesis is that temporal structures are sufficiently universal across domains for a model pre-trained on diverse data to generalize to new distributions [6].

One design challenge is the representation of continuous values. Unlike discrete tokens in language, time series values lie on a continuous scale with no fixed vocabulary. Among the strategies that have emerged [6], two are particularly relevant to the models evaluated in this work. *Tokenization-based* approaches discretize the values (for example by scaling and quantizing into a fixed set of bins) and then apply a standard language model architecture over the resulting token sequence. *Patch-based* approaches segment the input into fixed-length patches of consecutive observations, embed each patch as a single vector, and process the sequence of patch embeddings with a transformer. Both strategies reduce the effective sequence length relative to processing each observation individually. Prior work on applying these models to network traffic is surveyed in Section 2.3, and the specific models selected for evaluation are listed in Section 5.1.

1.4 Evaluation Measures for Forecasting

Forecasting accuracy is quantified by comparing predicted values $\hat{y}_t$ to observed values y_t over a forecast horizon of H steps beginning after T in-sample observations. The metrics below range from scale-dependent absolute errors to scale-free relative measures. Their properties and trade-offs motivate the metric selection in Section 5.4.

Mean Absolute Error (MAE).

$$\text{MAE} = \frac{1}{H} \sum_{t=T+1}^{T+H} |y_t - \hat{y}_t| \tag{1.4}$$

MAE averages the absolute deviations between predicted and observed values. It is scale-dependent: expressed in the same units as the data [34].

Root Mean Squared Error (RMSE).

$$\text{RMSE} = \sqrt{\frac{1}{H} \sum_{t=T+1}^{T+H} (y_t - \hat{y}_t)^2} \tag{1.5}$$

RMSE squares the errors before averaging, penalizing large deviations more heavily than MAE. Like MAE, it is scale-dependent [23].

Symmetric Mean Absolute Percentage Error (SMAPE). Multiple incompatible definitions of SMAPE exist in the literature [34]. The variant used in the M3 competition [35] places the sum of absolute values in the denominator:

$$\text{sMAPE} = \frac{1}{H} \sum_{t=T+1}^{T+H} \frac{200\,|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|} \tag{1.6}$$

This formulation is bounded between 0 and 200 and nominally scale-free [34].

Mean Absolute Scaled Error (MASE).

$$\text{MASE} = \frac{\dfrac{1}{H} \displaystyle\sum_{t=T+1}^{T+H} |y_t - \hat{y}_t|}{\dfrac{1}{T-m} \displaystyle\sum_{t=m+1}^{T} |y_t - y_{t-m}|} \tag{1.7}$$

where m is the seasonal period. MASE normalizes the forecast MAE by the in-sample error of the seasonal naïve forecast, making it scale-free and comparable across series [34]. A value below 1 indicates the model outperforms the seasonal naïve baseline. The specific sliding-window variant used in this work is described in Section 5.4.

Chapter 2
Existing Related Works

This chapter surveys prior work in three areas that motivate the contributions of this thesis: network traffic forecasting methods and the datasets used in their evaluation (Section 2.1), the evolution of time series benchmarking towards property-specific diagnostic evaluation (Section 2.2), and foundation models for time series and their applicability to network traffic (Section 2.3).

2.1 Network Traffic Forecasting

Network traffic forecasting has been studied for over four decades. Two recent surveys by Ferreira et al. [2] and Aouedi et al. [29] trace the progression from classical statistical methods through machine learning to deep learning approaches.

Statistical models, particularly ARIMA and its seasonal variant SARIMA, have been the dominant baseline in the field [2]. Their widespread adoption stems from interpretability and minimal data requirements, though they are limited to linear dependencies and require stationarity assumptions that network traffic frequently violates [3]. More recent work has applied deep learning models to network traffic, including recurrent architectures (LSTM, GRU) and transformer-based models [29]. Koumar et al. [22] conducted the most extensive comparison to date, evaluating multiple deep learning architectures on the CESNET-TimeSeries24 dataset across all three aggregation levels.

A persistent obstacle to progress in the field is the lack of shared evaluation infrastructure. Unlike domains such as electricity load or road traffic forecasting, which have standardized public datasets included in benchmark suites like the Monash Archive [13] or GIFT-Eval [14], network traffic forecasting relies overwhelmingly on private or restricted datasets [2]. The most frequently cited public datasets (Abilene [36], GÉANT [37], MAWI [38], CAIDA [39]) are old, small, often only partially released, and almost never reused with identical splits across studies [2]. As a result, no two recent papers in the surveyed

literature share both a dataset and an evaluation split, which prevents direct cross-paper comparison.

CESNET-TimeSeries24 [17] addresses the data availability problem by providing the first large-scale public ISP network traffic dataset under a permissive license, covering 275 124 individual IP addresses, 548 subnets, and 283 institutions over 40 weeks. It has since been adopted by Koumar et al. [22] for deep learning evaluation and by Liu et al. [11] for the first zero-shot evaluation of a foundation model (Tiny Time Mixers (TTM)) on ISP traffic. The dataset provides the raw material, but it is a collection of time series rather than a curated benchmark suite with defined selection criteria and evaluation protocols.

2.2 Time Series Benchmarking

Standardized benchmarks have been central to progress in time series forecasting. The M-competition series, from M4 [12] to M5 [16], established the practice of large-scale, reproducible model comparison with shared data splits and evaluation metrics. The Monash Time Series Forecasting Archive [13] extended this by aggregating 36 datasets across multiple domains into a unified repository. More recent benchmarks have expanded in both scale and scope. Table 2.1 summarizes representative benchmarks and their features.

TSLib [40] and TFB [41] broadened domain coverage and standardized evaluation protocols. GIFT-Eval [14] introduced per-domain breakdowns, reporting accuracy separately for each source domain rather than averaging across all series. fev-bench [42] added difficulty-based stratification by grouping series into easy, medium, and hard categories based on the performance of a reference model. TSFM-Bench [43] targeted foundation models specifically, comparing zero-shot, few-shot, and full-shot regimes. In the domain of road traffic, LargeST [44] provided a large-scale benchmark with hierarchical splits across geographic regions, and XXLTraffic [45] explicitly addressed distribution shift. The It's TIME benchmark [46] groups series post-hoc by interpretable temporal features, demonstrating that model rankings change substantially across feature clusters.

Two observations follow from Table 2.1. First, network traffic is absent from every existing general-purpose time series benchmark. Forecasting studies in the network traffic domain therefore cannot use the shared evaluation infrastructure that has driven reproducible progress in other domains. This work is the first benchmark to include ISP network traffic data.

Second, several recent benchmarks already stratify results by some grouping variable, as shown in the table. GIFT-Eval [14] reports per-domain breakdowns, fev-bench [42] stratifies by difficulty relative to a reference model, and It's TIME [46] groups series by temporal features, splitting at the population median of each feature so that the grouping changes with the dataset composition. Kang et al. [15] showed that different forecasting methods dominate

Table 2.1 Comparison of representative time series forecasting benchmarks. *Hierarchical Split* indicates whether results are reported separately across entity aggregation levels. *Behavioral Diagnostics* describes whether evaluation is stratified by structural time series properties. *Net. Traffic* marks inclusion of ISP or computer-network traffic series.

Benchmark	Year	Datasets	Series	Granularities	Hier. Split	Behav. Diagnostics	Net. Traffic
M4 [12]	2020	1	100 000	Hourly – Yearly	No	No	No
Monash [13]	2021	36	561 842	4-secondly – Yearly	No	No	No
M5 [16]	2022	1	42 840	Daily	Yes	No	No
TSLib [40]	2023	9	1 356	15-minutely – Weekly	No	No	No
LargeST [44]	2023	1	8 600	5-minutely	Yes	No	No
TFB [41]	2024	25	8 068	Minutely – Yearly	No	No	No
GIFT-Eval [14]	2024	24	144 000	10-secondly – Yearly	No	Per domain (post-hoc)	No
TSFM-Bench [43]	2025	21	5 909	Minutely – Yearly	No	Per training regime	No
XXLTraffic [45]	2025	10	19 555	5-minutely – Hourly	Yes	Per shift regime	No
fev-bench [42]	2026	96	55 627	Minutely – Yearly	No	Per difficulty (post-hoc)	No
It's TIME [46]	2026	50	2 386	Minutely – Yearly	No	Per pattern (post-hoc)	No
This work	**2026**	**1** (5 sub-bench.)	**532**	**10-min – Hourly**	**Yes**	**Per behavior (by constr.)**	**Yes**

in different regions of such a feature space, motivating finer-grained evaluation. This work takes a different approach: each benchmark selects series that pass a single fixed-threshold criterion, and several of these criteria are domain-specific to network traffic (e.g., working-hour ratio, drift detection between training and test splits). The five benchmarks and their selection pipelines are described in Chapter 4.

2.3 Foundation Models for Time Series

The foundation model paradigm is introduced in Section 1.3. While these models have achieved strong zero-shot performance on general-purpose benchmarks such as GIFT-Eval [14] and fev-bench [42], their evaluation on network traffic remains limited.

The published pre-training corpora of the foundation models evaluated in this work draw from domains such as energy, retail, finance, transportation, and web traffic [8, 9, 10, 47]. None of them include ISP-level network traffic data. Applying these models to network traffic therefore constitutes a genuine out-of-distribution test rather than an in-domain evaluation.

Liu et al. [11] provided the first such test, systematically evaluating TTM on CESNET-TimeSeries24 in both zero-shot and few-shot settings across all three aggregation levels. Their results showed that zero-shot TTM can outperform per-series trained deep learning baselines (GRU, LSTM) at the institution and subnet levels while maintaining low inference latency on commodity hardware. Performance degraded at the IP level, where extreme sparsity and irregular activity patterns reduced accuracy. Few-shot fine-tuning on small fractions of the training data yielded only marginal improvement over the zero-shot baseline. These findings establish that at least one foundation model generalizes to ISP traffic at coarser aggregation levels, but leave open the question of how different foundation models behave on series with specific structural properties.

This thesis addresses that gap by evaluating four foundation models on benchmarks that isolate individual traffic behaviors rather than averaging across all series.

Chapter 3

CESNET-TimeSeries24 Dataset

All five benchmarks and the subsequent evaluation are built on CESNET-TimeSeries24 [17], a public dataset of network-traffic time series captured over 40 weeks on the CESNET3 ISP network in the Czech Republic. The dataset has been used as the evaluation corpus in recent network-traffic forecasting work [22, 11]. This chapter summarizes the aspects of the dataset that are relevant to benchmark construction (Chapter 4) and evaluation (Chapter 5): data collection and access, scope, aggregation levels, feature set, and the statistical properties of the target variable `n_bytes`.

3.1 Data Collection and Access

The dataset originates from IPFIX flow records exported by `ipfixprobe` from monitoring probes placed before the edge routers of CESNET3, which interconnects Czech academic and research institutions. All transit flows (where both endpoints lie outside CESNET3) and single-packet TCP-SYN scans are filtered out before aggregation to remove noise that would otherwise dominate the signal. Each flow record contains only header-level metadata (source and destination IP addresses, transport ports, protocol, packet and byte counts, TTL, timestamps). No payload is stored, and IP addresses, institution names, and subnet identifiers are replaced with sequential random integers at ingest. The publication of the dataset was approved by the Committee for Ethics in Research of the CTU Scientific Council [17].

The dataset is distributed as compressed Comma-Separated Values (CSV) files via Zenodo [17]. In this work, it is accessed through the CESNET TS-Zoo library [48], which provides programmatic loading, filtering, and aggregation-level selection and is used consistently both in the benchmark selection pipelines (Chapter 4) and in the evaluation framework (Chapter 6).

3.2 Scope, Time Range, and Resolution

Capture began on 9 October 2023 and ended on 14 July 2024, spanning 40 weeks of continuous monitoring. During that period, the probes processed approximately 66 billion IP flows comprising 4 trillion packets and 3.7 petabytes of data. The dataset is provided at three temporal resolutions: 10-minute, 1-hour, and 1-day aggregations.

One known artifact is relevant to any long-horizon analysis: one monitoring probe was inoperative from approximately 21 May 2024, 16:30 to 4 June 2024, 20:00, producing a dataset-wide drop in traffic volume during that window. Because this gap is systemic rather than entity-specific, it influences many series simultaneously and is retained as captured rather than imputed. [17]

3.3 Aggregation Levels

CESNET-TimeSeries24 organizes series at three hierarchical scopes: individual IP addresses, institution subnets (geographically or organizationally distinct networks within a single institution), and whole institutions [17]. Table 3.1 summarizes the counts.

Table 3.1 CESNET-TimeSeries24 aggregation levels. Counts refer to active series after the original authors' filtering step that discarded series with more than 99.75 % empty 10-minute intervals [17]. The 1 000-series IP sample (`IP_ADDRESSES_SAMPLE`) is a pre-selected subset provided alongside the full IP dataset.

Level	Count	Scope
Institutions	283	Aggregate traffic of all IP addresses belonging to one academic or research institution
Institution subnets	548	Subnetworks within institutions, typically distinguishing locations or organizational units
IP addresses (full)	275 124	Individual endpoints active during the capture window
IP addresses (sample)	1 000	Pre-selected subset of the full IP set, used for initial filtering passes

Aggregation scope directly influences the statistical structure of a series. At the institution level, traffic is the sum over hundreds or thousands of endpoints, so short bursts average out and daily and weekly periodicity dominate [26]. At the IP level, individual device behavior is preserved, producing a heterogeneous population ranging from always-on servers to sporadically active

workstations [17]. The five benchmarks exploit this heterogeneity by drawing from the level at which the targeted property is meaningful (e.g., workstation working hours are visible only at the IP level, as the pattern washes out under aggregation).

3.4 Feature Set

Each 10-minute or 1-hour datapoint carries twelve metrics grouped by semantic role [17], summarized in Table 3.2. The 1-hour and 1-day aggregations additionally expose sum, mean, and standard deviation variants of the three *unique* volumetric metrics, bringing the column count to 18.

Table 3.2 Per-datapoint metrics in CESNET-TimeSeries24 [17]. Ratios are bounded in $[0, 1]$ by construction.

Group	Metrics
Simple volumetric	`n_flows`, `n_packets`, `n_bytes`
Unique volumetric	`n_dest_ip`, `n_dest_asn`, `n_dest_port`
Ratios	`tcp_udp_ratio_packets`, `tcp_udp_ratio_bytes`, `dir_ratio_packets`, `dir_ratio_bytes`
Averages	`avg_duration`, `avg_ttl`

The target variable for all subsequent analysis is `n_bytes`, the total number of bytes transmitted within the aggregation interval. It measures bandwidth usage and network load, and it is used as a primary target in prior network-traffic forecasting work on this dataset [22, 11]. Both the 1-hour and 10-minute aggregations are used in this work, while the 1-day resolution is excluded because it collapses the daily and weekly periodicities on which four of the five benchmarks rely [26].

3.5 Properties of `n_bytes`

Two properties of `n_bytes` shape the statistical analyses and modeling decisions in later chapters, namely a heavily right-skewed distribution and a level-dependent sparsity. The statistics below are reported for the hourly resolution. The same patterns hold at the 10-minute resolution and are reported in Appendix A.

3.5.1 Sparsity

The dataset stores intervals during which a monitored device does not transmit any data as missing (NaN) values [17]. Because `n_bytes` measures a physical byte count, these gaps represent intervals where no traffic was transmitted

and a zero value carries true information [22]. All missing values are therefore replaced by zero before any analysis in this work. The term *sparsity* refers to the fraction of such zero-valued observations.

As noted in Section 3.3, aggregation scope determines how many non-zero observations a series contains. Table 3.3 reports the mean and median percentage of non-zero datapoints at each level. Institutions and subnets are nearly fully populated, whereas the median IP series contains only 7.4 % non-zero observations [17].

Table 3.3 Percentage of non-zero datapoints (mean and median across all series at each aggregation level).

Level	Mean (%)	Median (%)
Institutions	98.9	99.99
Subnets	96.1	99.99
IP addresses	21.0	7.4

This sparsity at the IP level explains the residual skewness after the log transform noted in Section 3.5.2 and motivates the per-benchmark sparsity filters applied in Chapter 4.

3.5.2 Distribution

Individual `n_bytes` series are heavily right-skewed at all three aggregation levels. Table 3.4 reports the median of three per-series statistics across all series at each level.

Table 3.4 Median of per-series distribution statistics at each aggregation level, computed on raw `n_bytes` and on log-transformed values $\tilde{x}_t = \log(1 + x_t)$.

Level	Transform	Skewness	Kurtosis	Max/Median
Institutions	raw	4.9	45.2	93.6
	transformed	−0.7	5.1	1.3
Subnets	raw	6.3	78.5	124.9
	transformed	−0.7	5.7	1.3
IPs	raw	28.0	987.6	1111.6
	transformed	3.4	15.8	1.7

The skew increases with disaggregation. At the institution level, aggregation over many endpoints smooths out individual bursts, yielding a median skewness of 4.9. At the IP level, single-device behavior is preserved and the median skewness rises to 28, with excess kurtosis near 1,000. In a typical IP-level series the maximum observation exceeds the median by three orders of magnitude. The levels also differ in absolute magnitude: the median of

per-series mean hourly `n_bytes` is 2.3×10^8 at the institution level, 7.9×10^7 at the subnet level, and 7.6×10^5 at the IP level—more than two orders of magnitude between the highest and lowest levels.

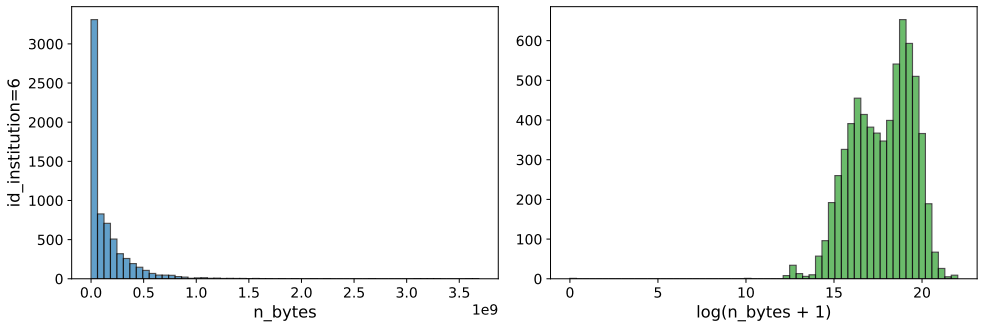


Figure 3.1 Histogram of `n_bytes` for a representative institution-level series (selected as the series closest to the median skewness). Left: raw values (skewness 4.86); right: after $\log(1 + x_t)$ transform (skewness -0.52).

The transform $\tilde{x}_t = \log(1 + x_t)$ substantially reduces this skewness (Table 3.4, Figure 3.1), bringing the median below 2 at all levels. The residual skew at the IP level is attributable to the high zero fraction (Section 3.5.1). The implications for model preprocessing are discussed in Section 5.3. The same transform is used in STL decomposition for the seasonality benchmark [23].

Chapter 4
Benchmarks

Forecasting models are commonly evaluated on large, heterogeneous benchmark collections [12, 13, 14], which aggregate performance metrics across thousands of series from diverse domains and temporal resolutions into a single leaderboard ranking. While this breadth provides a broad view of general forecasting competence, it has a fundamental limitation: accuracy averaged over structurally different series can mask systematic failures on particular types of data. Kang et al. [15] demonstrated this explicitly by mapping time series into a multidimensional feature space and showing that different forecasting algorithms dominate in different regions. As a result, a model that ranks first overall may still perform poorly on series with a specific structure.

Rather than aggregating all patterns into a single larger mixed collection, this work constructs five *property-specific diagnostic benchmarks*, each guaranteeing that every selected series exhibits one predefined structural characteristic. When the property is controlled by construction, accuracy differences between models can be directly attributed to their ability to exploit or cope with that characteristic.

4.1 Overview

The five benchmarks are:

1. **Seasonality Benchmark** — series with strong recurring daily and/or weekly periodic patterns in byte-level traffic volume.
2. **Data Drift Benchmark** — series exhibiting a sustained, directional shift in traffic level, with the shift guaranteed to fall within the held-out test portion of each series.
3. **Periodic Spaces Benchmark** — IP-level series in which traffic follows a regular on/off cycle, alternating between active and silent periods at a consistent hourly interval.

4. **Workstation Activity Benchmark** — IP-level series active primarily during weekday business hours, reflecting a human-operated device following an office schedule.

5. **No Temporal Structure Benchmark** — series that have no statistically meaningful autocorrelation for a model to exploit; this benchmark is a diagnostic check that models do not overfit to noise.

All benchmarks use `n_bytes` over the full available time range at two temporal resolutions: hourly (`AGG_1_HOUR`, 6 718 observations) and 10-minute (`AGG_10_MINUTES`, 40 298 observations). This chapter describes the hourly pipeline. The 10-minute resolution uses scaled parameters as discussed in Section 4.2. The choice of target variable and temporal resolution is motivated in Section 3.4.

Depending on the property targeted, each benchmark covers one or more of the three aggregation levels: Institutions (283 series), Institution Subnets (548 series), and IP Addresses. For IP-level series, candidates are initially drawn from a random sample of 1 000 addresses. If the qualifying pool from this sample is too small to reach the target of 25 series, the pipeline is extended to the full set of over 275 000 active IPs [17]. Shared selection methodology is described in Section 4.2. Individual pipelines are detailed in Sections 4.3–4.7.

4.2 Common Methodology

Sparsity filter. Before any benchmark-specific analysis, each series is checked for excessive sparsity. The activity ratio is defined as:

$$\text{ratio_active} = \frac{\#\{t : \texttt{n_bytes}(t) > 0\}}{T},$$

where T is the total number of hourly observations. Series that fall below a benchmark-specific activity threshold are discarded because near-zero series produce uninformative statistics (see Section 1.2 for a detailed discussion). The exact threshold and its rationale are stated in each benchmark section. Where a qualifying pool exceeds 25 series, 25 are sampled uniformly at random and `random_state=42` is used throughout for reproducibility.

10-minute resolution. The entire selection pipeline is also executed at the 10-minute aggregation level (`AGG_10_MINUTES`). All parameters expressed in hourly units are scaled by a factor of 6 to preserve equivalent time spans. The methodology is otherwise identical. Pool sizes differ because the finer granularity changes sparsity statistics and autocorrelation structure. The resulting series selections for both resolutions are listed in Appendix B.

Data format and storage. Each benchmark is stored in two formats. The long-format CSV—one row per timestamp–series pair—is provided for interoperability and human readability. Apache Parquet, a column-oriented binary format, is provided alongside the CSV export: it offers superior compression and is a storage format adopted by state-of-the-art time series benchmarks and deep learning frameworks [14, 42].

4.3 Seasonality Benchmark

Network traffic volume often exhibits a two-period structure: a daily cycle driven by working-hour activity and a weekly cycle reflecting the weekday–weekend contrast [26, 17]. This pattern is most pronounced at the institution and subnet level, but also appears in a subset of individual IP addresses. This benchmark selects series that are strongly governed by periodic seasonality.

Sparsity filter. Series with fewer than 40 % of non-zero hours are discarded (ratio_active < 0.4, applied at all three aggregation levels). STL decomposition is uninformative when most observations are zero (Section 1.2). After filtering, 281 Institutions, 535 Subnets, and 183 IP series remain as candidates.

Seasonal strength scoring. Before decomposition, each series is transformed with $\tilde{x}_t = \log(1 + x_t)$ to prevent extreme traffic spikes from inflating $\mathrm{Var}(R)$ and suppressing the seasonal strength score (see Section 3.5.2). The seasonal strength F_S (Equation 1.3) is then computed independently for the two natural periods present in hourly network data: daily ($m = 24\,\mathrm{h}$) and weekly ($m = 168\,\mathrm{h}$). The final score per series is

$$\text{max_strength} = \max\bigl(F_S^{(24)},\ F_S^{(168)}\bigr).$$

Only series with max_strength ≥ 0.7 pass the threshold. Table 4.1 shows the pool sizes at each aggregation level. 25 series are then drawn uniformly at random from each pool. For IPs, the qualifying pool from the 1 000-series sample (8 series) was too small, so the pipeline was re-run on the full IP dataset (275 124 series) and 25 series were sampled from the resulting pool of 1 413. The 75 randomly sampled series are listed in Appendix B. Figure 4.1 shows the STL decomposition for Institution 15, illustrating how the seasonal component dominates the residual.

Figure 4.1 STL decomposition for Institution 15. From top to bottom: log-transformed observed series, trend, seasonal component, and residual. The seasonal component captures the dominant weekly cycle; its large variance relative to the residual yields a high seasonal strength score.

Table 4.1 Seasonality benchmark: candidate counts at each filtering stage.

Level	Total	After sparsity filter	Qualifying pool ($\geq$0.7)
Institutions	283	281	145
Subnets	548	535	176
IPs	275 124	42 706	1 413

4.4 Data Drift Benchmark

Network traffic levels are non-stationary over multi-month horizons: changes in network configurations, user behaviors, or network conditions can permanently alter the traffic of an institution, subnet, or host [29]. When such a structural level shift persists into the test window, a model extrapolating from the pre-drift baseline will systematically mis-forecast, producing errors that in-sample tuning cannot remove. This benchmark isolates sustained level shifts from regular seasonal patterns and transient spikes. It covers all three aggregation levels.

Sparsity filter. Distinguishing a genuine level shift from sparsity-induced fluctuations requires a nearly uninterrupted signal. The sparsity threshold is therefore stricter than for the Seasonality benchmark: ratio_active ≥ 0.9. Table 4.2 shows the number of series passing this filter at each aggregation level.

Relative deviation between moving averages. For each candidate series, two rolling moving averages are computed on the raw `n_bytes` values: a short-term window of 168 h (one calendar week) tracking recent behavior, and a long-term window of 744 h (31 days) approximating the established baseline. The divergence between the two is expressed as a relative deviation:

$$\text{rel_dev}(t) = \frac{\left|\text{MA}_{\text{short}}(t) - \text{MA}_{\text{long}}(t)\right|}{\text{MA}_{\text{long}}(t)}. \tag{4.1}$$

Normalizing by the long-term Moving Average (MA) makes the score volume-independent: a deviation of 0.30 means that recent traffic is 30 % above or below the one-month baseline, regardless of absolute traffic volume.

A single spike above the threshold may reflect a transient burst rather than a structural change. A *drift event* is therefore defined as a consecutive period in which $\text{rel_dev}(t) > 0.30$ persists for at least 14 consecutive days (336 consecutive hours). Furthermore, detecting drift solely within the training window is insufficient: if the series reverts to its prior level before the test split, a model trained on the drifted portion may still generalize accurately. The benchmark therefore additionally requires that at least one timestep of a drift event falls within the test portion—the final 20 % of observations. Only series satisfying both conditions qualify for selection. Figure 4.2 illustrates the full detection pipeline on Institution 209. The resulting pool sizes are shown in Table 4.2. 25 series are drawn uniformly at random from each qualifying pool. The 75 randomly sampled series are listed in Appendix B.

Table 4.2 Data Drift benchmark: candidate counts at each filtering stage.

Level	Total	After sparsity filter	Has drift	Qualifying pool
Institutions	283	276	134	103
Subnets	548	507	289	214
IPs	1 000	95	67	49

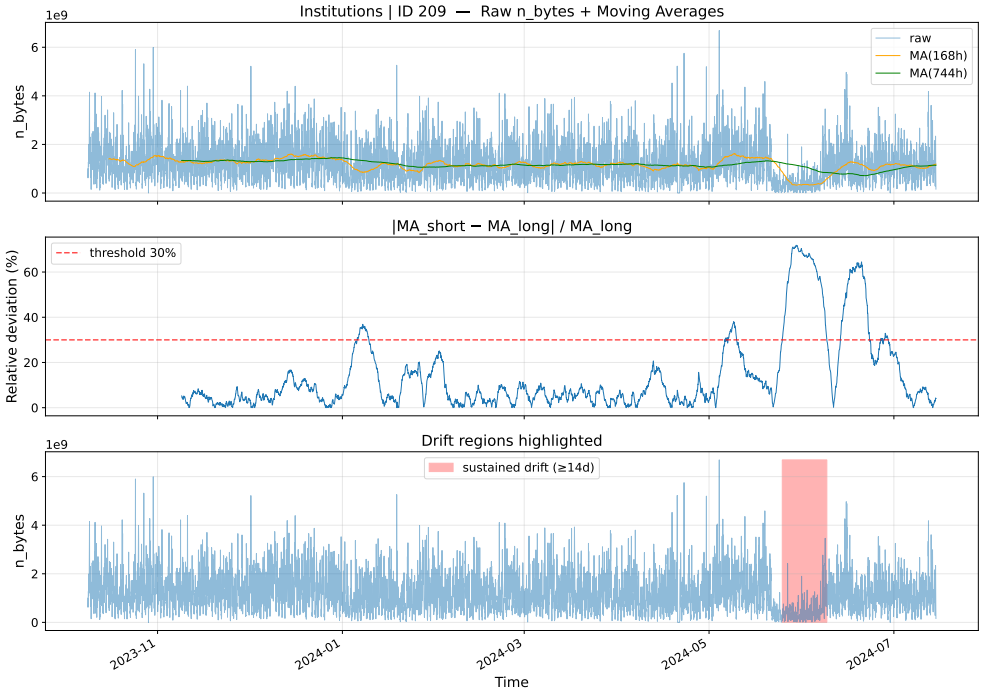


Figure 4.2 Drift detection for Institution 209. Top: raw `n_bytes` with the short-term MA (168 h, orange) and long-term MA (744 h, green). Middle: relative deviation between the two averages; Bottom: raw series with sustained drift regions (≥336 consecutive hours above threshold) highlighted in red.

4.5 Periodic Spaces Benchmark

Some IP-level network endpoints transmit in regular on/off cycles: a backup job that runs nightly, a device that polls a remote server once a week, or an automated task that uploads data at a fixed daily interval. The resulting `n_bytes` series is a continuous traffic-volume signal with structured gaps. A forecasting model that captures this periodicity must predict not only the traffic volume during active periods but also when the endpoint will be completely silent. This benchmark is restricted to IP-level series because at the Institution or Subnet level, traffic from many individual endpoints is summed and the on/off cycles of different devices do not align, so the aggregated signal does not contain periodic gaps.

Unlike the other benchmarks, which apply a one-sided lower bound on activity, the Periodic Spaces benchmark applies a two-sided sparsity band: $0.05 \leq \text{ratio_active} \leq 0.95$. A series that is nearly always active or inactive has no meaningful zero/non-zero alternation to detect. After filtering, 473 of the

1 000 IP series pass. The selection criterion targets the *timing* of transmission, not the volume. For each candidate series a binary indicator is derived:

$$B_t = \begin{cases} 1 & \text{if } \texttt{n_bytes}(t) = 0 \quad \text{(inactive hour)}, \\ 0 & \text{if } \texttt{n_bytes}(t) > 0 \quad \text{(active hour)}. \end{cases}$$

The sample autocorrelation of B_t is computed for lags 1 through 336 (two calendar weeks).

Peak-based periodicity detection. Taking the highest ACF value excluding lag 0 is unreliable: positively autocorrelated series exhibit monotonic decay from the zero lag, causing the global maximum to default to lag 1 [3]. True periodicity instead appears as a local peak rising above the post-decay trough at seasonal intervals [3]. Seasonal peaks are identified using `scipy.signal.find_peaks` with a prominence threshold of 0.1. Because topographic prominence measures a peak's height relative to its nearest saddle point rather than its absolute value [49], minor fluctuations along the initial ACF decay receive low prominence scores, so only true periodic peaks that rise clearly above the post-decay trough are detected.

From the detected peaks, the one with the highest ACF value (max_acf) is selected. The corresponding lag defines dominant_lag (period length in hours). If no peaks are found, max_acf is set to 0. Only series with max_acf ≥ 0.6 are admitted to the qualifying pool. Of the 473 candidates, 25 pass this threshold, therefore all 25 are selected. They are listed in Appendix B. 16 exhibit weekly periodicity (dominant_lag = 168 h), 8 daily (24 h), and 1 near-daily (23 h). Figure 4.3 illustrates the detection pipeline for IP 722 293.

Table 4.3 Periodic Spaces benchmark: candidate counts at each filtering stage.

Level	Total	After sparsity band	Qualifying pool ($\geq$0.6)
IPs	1 000	473	25

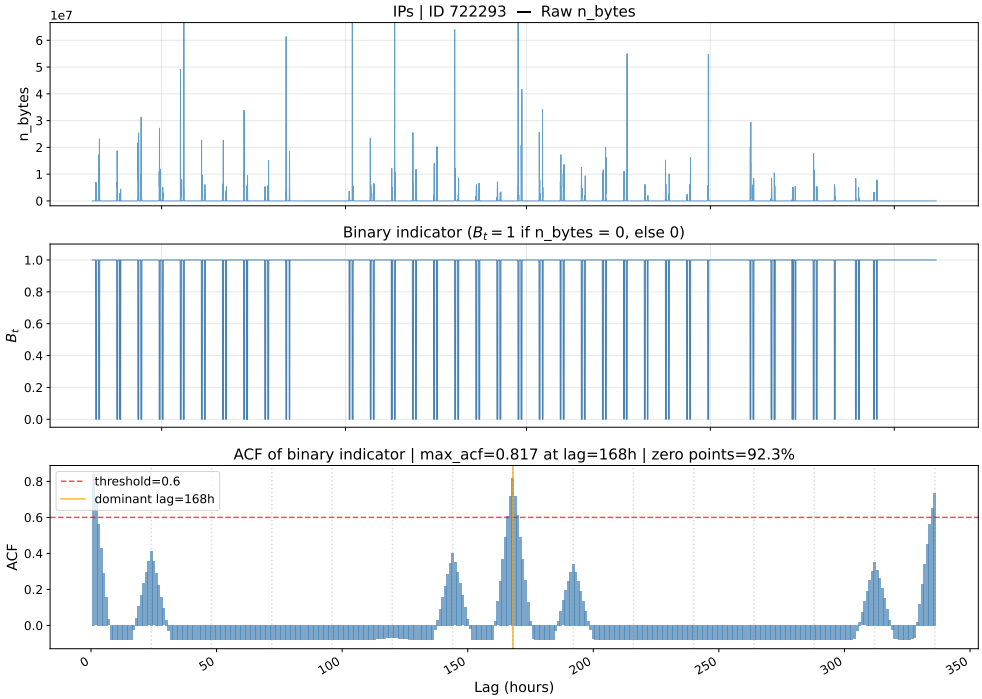


Figure 4.3 Periodic spaces detection for IP 722 293. Top: raw `n_bytes` (y-axis clipped at the 99th percentile) with visible on/off blocks. Middle: binary indicator B_t (1 if inactive, 0 if active). Bottom: ACF of B_t for lags 1–336; the dominant peak at lag 168 h (weekly) exceeds the 0.6 threshold. Gray dotted lines mark 24 h multiples.

4.6 Workstation Activity Benchmark

Individual office workstations generate network traffic that closely follows the human working schedule: activity is concentrated on weekdays during business hours and drops to near-zero in evenings, on weekends, and on public holidays [17]. Unlike the statistical seasonality captured by the Seasonality benchmark, this periodic structure is driven by an external human schedule rather than by recurring patterns inherent in the traffic data. Accurate forecasting therefore requires the model to incorporate calendar context (day-of-week, hour-of-day, holiday status) instead of relying solely on recent traffic levels. This benchmark is restricted to IP-level series. At the Institution or Subnet level, traffic from many heterogeneous devices is aggregated, mixing workstations with always-on infrastructure, and the business-hours concentration is then washed out.

Series with ratio_active ≤ 0.23 are discarded. This threshold corresponds to approximately 1 545 active hours out of 6 718 total—roughly 39 working

weeks of 40 hours each. For each remaining series, the Working Hours Ratio (WHR) is computed as

$$\mathrm{WHR} = \frac{\#\{t : \texttt{n_bytes}(t) > 0 \ \wedge \ t \in \mathcal{W}\}}{\#\{t : \texttt{n_bytes}(t) > 0\}}, \tag{4.2}$$

where the working-hour set $\mathcal{W}$ contains every hour t that falls on a Monday–Friday between 08:00 and 18:59 (inclusive) and is not a public holiday. Public holidays are sourced from the `weekends_and_holidays` metadata provided with the CESNET-TimeSeries24 dataset [17]. The denominator is the total number of active hours; $\mathrm{WHR} \in [0, 1]$, where $\mathrm{WHR} = 1$ indicates that all traffic falls within working hours. Only series with $\mathrm{WHR} \geq 0.6$ qualify. Of the 1 000 candidate IP series, 236 pass the sparsity filter and 1 meets the WHR criterion. The pipeline was therefore re-run on the full IP dataset (275 124 series) and 25 series were sampled from the resulting pool of 120. Pool counts are shown in Table 4.4. The 25 randomly sampled series are listed in Appendix B. Figure 4.4 shows IP 21 943 as a representative example.

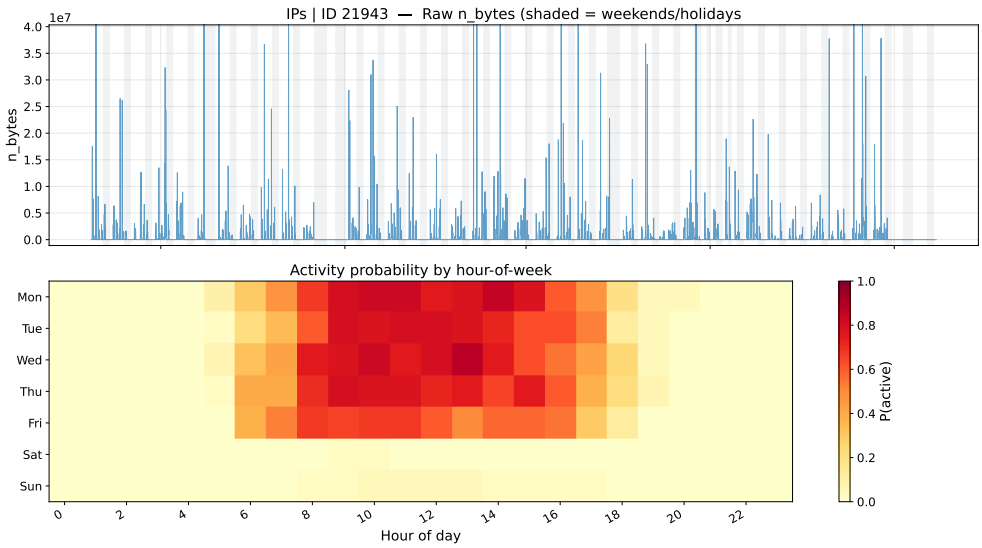


Figure 4.4 Workstation activity pattern for IP 21 943. Top: raw `n_bytes` (y-axis clipped at the 99th percentile); shaded regions mark weekends and public holidays. Bottom: activity probability heatmap by hour-of-day and day-of-week, showing traffic concentrated on weekdays between 08:00 and 19:00.

Table 4.4 Workstation Activity benchmark: candidate counts at each filtering stage.

Level	Total	After sparsity filter	Qualifying pool ($\geq$0.6)
IPs	275 124	60 022	120

4.7 No Temporal Structure Benchmark

This benchmark tests whether models default to a naive prediction when no temporal pattern is present, rather than overfitting to noise. It selects series that behave as close to white noise as possible, with no significant autocorrelation at any lag up to one week. Unlike the other four benchmarks, which target specific exploitable patterns, this benchmark establishes a lower bound on expected model performance. It covers all three aggregation levels, because structureless series can arise at any level of aggregation.

Very sparse series are dominated by zeros. Their apparent randomness is trivially caused by the binary on/off pattern rather than genuine variability in the traffic values. Series with ratio_active < 0.4 are therefore discarded. Counts after this filter are shown in Table 4.5. For each remaining series the maximum absolute autocorrelation across lags 1 through 168 is computed:

$$\text{max_ACF} = \max_{k=1,\ldots,168} |\text{ACF}(k)|,$$

where $\text{ACF}(k)$ is the sample autocorrelation at lag k, calculated on the raw (untransformed) `n_bytes` series. A single statistic over all 168 lags captures any periodic (daily, weekly) or persistent structure regardless of the specific lag at which it appears.

Series with max_ACF ≤ 0.1 form the qualifying pool. The pool sizes are shown in Table 4.5. 25 series are drawn uniformly at random from each pool. If the qualifying pool contains fewer than 25 series, all qualifying series are selected. For Institutions, the pool of 13 is smaller than the target, so all 13 are selected. The 63 randomly sampled series (13 Institutions, 25 Subnets, 25 IPs) are listed in Appendix B. Figure 4.5 shows Subnet 89 as a representative example: the ACF values across all 168 lags remain within the ± 0.1 threshold.

Table 4.5 No Temporal Structure benchmark: candidate counts at each filtering stage.

Level	Total	After sparsity filter	Qualifying pool ($\leq$0.1)
Institutions	283	281	13
Subnets	548	535	50
IPs	1 000	183	49

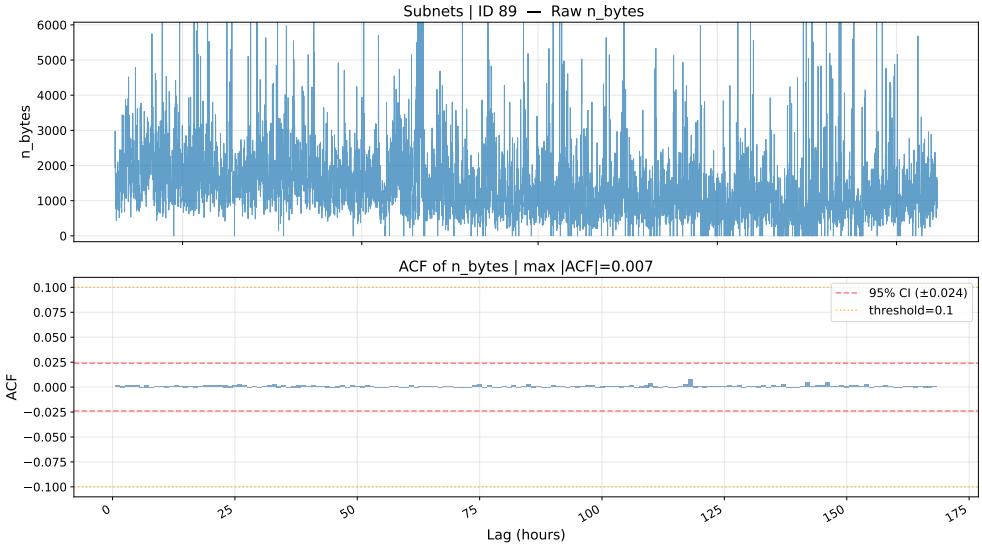


Figure 4.5 ACF verification for Subnet 89. Top: raw `n_bytes` (y-axis clipped at the 99th percentile). Bottom: sample ACF for lags 1–168; all values fall within the ±0.1 selection threshold (orange) and the 95 % confidence interval (red dashed).

4.8 Summary

Table 4.6 consolidates the five benchmarks at both temporal resolutions. The hourly resolution comprises 263 series and the 10-minute resolution 269 series, both spanning all three aggregation levels. The only difference is the No Temporal Structure benchmark, where finer granularity increases autocorrelation noise and raises the number of qualifying Institution series from 13 to 19. Each benchmark targets a single structural property through its selection criteria.

The five benchmarks are designed to test different model capabilities. Seasonality and Periodic Spaces both target repeating patterns, but in different ways: Seasonality selects series with smooth recurring traffic cycles (at all three aggregation levels), while Periodic Spaces captures binary on/off activity patterns visible only at IP level. Workstation Activity tests whether models can use a known external schedule (weekday business hours) rather than learned autocorrelation. Data Drift checks robustness to distribution shift by requiring a sustained deviation between short- and long-term traffic levels that extends into the test window. No Temporal Structure serves as a sanity check—series with near-zero autocorrelation. Evaluating a model across all five benchmarks gives a detailed picture of which traffic properties it captures well and which remain problematic.

Out of the 263 hourly series slots, only 10 appear in two benchmarks simultaneously — 7 shared between Seasonality and Data Drift (dense continuous series satisfy both criteria) and 3 between Data Drift and No Temporal Structure (level shifts without periodic autocorrelation). The 10-minute resolution

Table 4.6 Number of time series selected into each diagnostic benchmark at both temporal resolutions. Inst. = Institutions, Sub. = Subnets. †No Temporal Structure selects all qualifying Institution series (pool < 25): 13 at hourly, 19 at 10-minute.

Benchmark	Agg.	Criterion	Inst.	Sub.	IPs	Total
Seasonality	10 min	$\max(F_S^{(144)}, F_S^{(1008)}) \geq 0.7$	25	25	25	75
	1 h	$\max(F_S^{(24)}, F_S^{(168)}) \geq 0.7$	25	25	25	75
Data Drift	10 min	$\delta > 0.30$ for $\geq$2016 lags, overlaps test	25	25	25	75
	1 h	$\delta > 0.30$ for $\geq$336 lags, overlaps test	25	25	25	75
Per. Sp.	10 min	$\text{max_acf}(B_t) \geq 0.6$	–	–	25	25
	1 h	$\text{max_acf}(B_t) \geq 0.6$	–	–	25	25
Workstation	10 min	WHR ≥ 0.6	–	–	25	25
	1 h	WHR ≥ 0.6	–	–	25	25
No Temp.†	10 min	maxACF ≤ 0.1 for lags 1–1008	19	25	25	69
	1 h	maxACF ≤ 0.1 for lags 1–168	13	25	25	63
Total	10 min		69	75	125	269
Total	1 h		63	75	125	263
		Total	**132**	**150**	**250**	**532**

exhibits the same two overlap pairs. The overlap between Drift and No Temporal Structure is 8 series, consistent with finer granularity introducing more noise that lowers autocorrelation. The overlap between Seasonality and Drift is 4 series. Periodic Spaces and Workstation Activity remain fully disjoint from all other benchmarks at both resolutions.

Chapter 5
Evaluation Methodology

This chapter specifies the experimental setup used to evaluate forecasting models on the five benchmarks defined in Chapter 4. It first introduces the selected models, then describes the data preparation and sliding-window evaluation protocol, the per-series preprocessing applied to the deep learning models, the accuracy metrics, and finally the result aggregation and ranking procedure.

5.1 Selected Forecasting Models

The evaluation covers ten models: one trivial baseline (Seasonal Naïve), two classical statistical models (SARIMA, Prophet), four pre-trained foundation models applied zero-shot (TimesFM 2.5, TTM, Chronos-2, Moirai 2.0-R-Small), and three recurrent deep learning models trained individually per series (GRU, LSTM, GRU-FCN). The selection spans the full cost–accuracy spectrum, from a naïve reference requiring only a context buffer to supervised models fitted individually per series at high computational cost. Each model's architecture is described in Section 1.3 and its implementation in Chapter 6. The list below summarizes the selection rationale and relevant configuration for evaluation.

Seasonal Naïve. Repeats the last seasonal cycle from the context window: $\hat{y}_{t+h} = y_{t+h-m}$, where m is the seasonal period ($m = 24$ for hourly data, $m = 144$ for 10-minute data). It serves as a lower-bound baseline that any reasonable forecasting model should outperform.

SARIMA. Seasonal ARIMA [3] is the dominant statistical baseline in the network traffic forecasting literature [2, 22] and serves the same role here. It is configured with fixed order $(p, d, q)(P, D, Q)_S = (1, 1, 1)(1, 1, 1)_S$, where the seasonal period is $S = 24$ for hourly data (daily periodicity) and $S = 6$ for 10-minute data (hourly periodicity). Using the daily period $S = 144$

at 10-minute resolution would be computationally prohibitive, as SARIMA estimation scales with S and refitting occurs at every sliding window.

Prophet. A general-purpose forecasting tool [30] that decomposes series into trend, seasonality, and holiday components. It is included to test whether its built-in multi-seasonality and holiday handling generalize to network traffic patterns. Daily and weekly seasonality are left at their default auto-detection, and the Czech Republic holiday calendar is added to reflect the Czech academic origin of the CESNET network. Yearly seasonality is explicitly disabled.

TimesFM 2.5. A 200 M-parameter foundation model [8] that ranks among the top zero-shot forecasting models on both the GIFT-Eval and fev-bench benchmarks [14, 42]. Applied zero-shot.

TTM (Tiny Time Mixers). At only 1 M parameters, TTM [47] represents the compact end of the foundation model spectrum. It has already been evaluated zero-shot on CESNET-TimeSeries24 by Liu et al. [11], providing a direct point of comparison. TTM checkpoints fix the context length at training time, so the checkpoint revision is selected per configuration to match the required context and horizon. The effective context length is read from the checkpoint and overrides the value in Table 5.1.

Chronos-2. A probabilistic foundation model [9] that achieves state-of-the-art zero-shot accuracy on both GIFT-Eval and fev-bench [14, 42]. Applied zero-shot; because Chronos-2 outputs a full quantile distribution, the median is taken as the point forecast.

Moirai 2.0-R-Small. A probabilistic foundation model [10] that complements Chronos-2 with a different architecture (decoder-only with quantile regression instead of tokenized outputs) and a different pre-training corpus. Applied zero-shot; as with Chronos-2, the median quantile is used for the point forecast.

GRU, LSTM, and GRU-FCN. Three recurrent architectures trained per series, adopted directly from Koumar et al. [22] who evaluated them on the same CESNET-TimeSeries24 dataset. Their inclusion enables a direct comparison with prior work and represents the supervised per-series deep learning paradigm. The architectures and hyperparameters (number of RNN layers, hidden size, learning rate, batch size, number of epochs) are taken from Koumar et al. All three share a common preprocessing pipeline and training procedure detailed in Section 5.3.

All four foundation models apply per-window normalization internally during their forward pass (mean–standard-deviation standardization for TimesFM, TTM, and Moirai; mean–standard-deviation standardization followed by an arcsinh transform for Chronos-2). Foundation models are therefore passed

raw `n_bytes` values with no external preprocessing. Applying an additional external scaler would normalize the input twice and create a distribution mismatch relative to what the models observed during pre-training.

5.2 Data Preparation and Evaluation Protocol

As described in Section 3.5.1, missing values in the dataset are replaced by zero because they represent intervals with no traffic.

All benchmark series described in Chapter 4 (263 at hourly resolution, 269 at 10-minute resolution) are split chronologically into three non-overlapping segments: training (60 %), validation (20 %), and test (20 %). The split boundaries are computed per series as $t_{\text{train}} = \lfloor 0.6\,N \rfloor$ and $t_{\text{val}} = t_{\text{train}} + \lfloor (N - t_{\text{train}})/2 \rfloor$, where N is the series length. The training and validation sets are used for models that require supervised training for weight estimation. The test set is held out entirely for evaluation.

Models are evaluated under four context/horizon configurations across two temporal resolutions (Table 5.1). The hourly context and horizon lengths align

Table 5.1 Evaluation configurations. C denotes the number of past observations provided to the model; H denotes the number of steps forecast at once. Values are in timesteps (1 h for hourly, 10 min for 10-minute resolution), with equivalent durations in parentheses.

Configuration	Resolution	Context C	Horizon H
Short	Hourly	168 (7 d)	24 (1 d)
Long	Hourly	744 (31 d)	168 (7 d)
Short	10-minute	288 (2 d)	144 (1 d)
Long	10-minute	1,008 (7 d)	144 (1 d)

with daily and weekly intervals widely used in network traffic forecasting research [2, 29]. Both 10-minute configurations forecast 24 hours ahead (144 steps), matching the hourly short horizon in duration but at 6× finer granularity. Their context lengths of 2 and 7 days test whether additional history improves accuracy at this resolution. The four configurations together probe model sensitivity to context length, forecast horizon, and temporal resolution. Throughout the rest of this thesis, each configuration is referred to by the shorthand resolution/C/H, where h denotes hourly and m denotes 10-minute resolution. For example, h/168/24 is the short hourly configuration.

Forecasting accuracy on the test set is measured using a non-overlapping rolling-origin scheme [50]. Let T denote the length of the test set. The procedure generates $W = \lceil T/H \rceil$ forecast windows as follows:

1. The initial context is set to the last C observations of the combined training and validation sets.

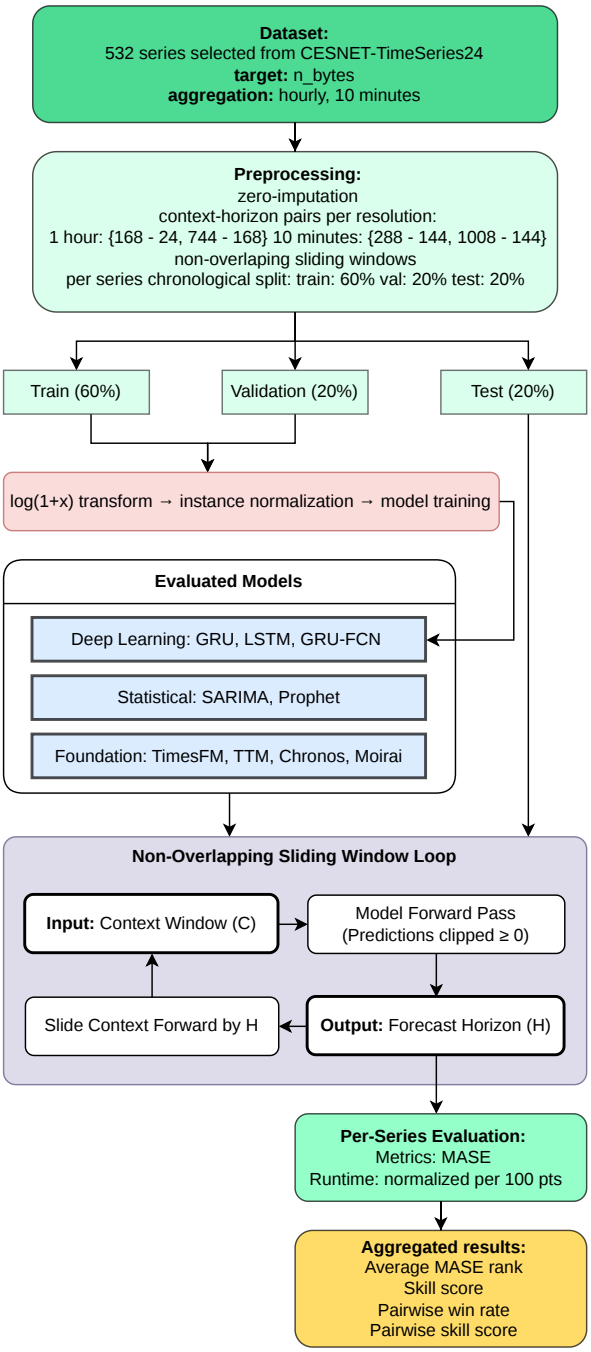


Figure 5.1 Overview of the data preparation and evaluation protocol.

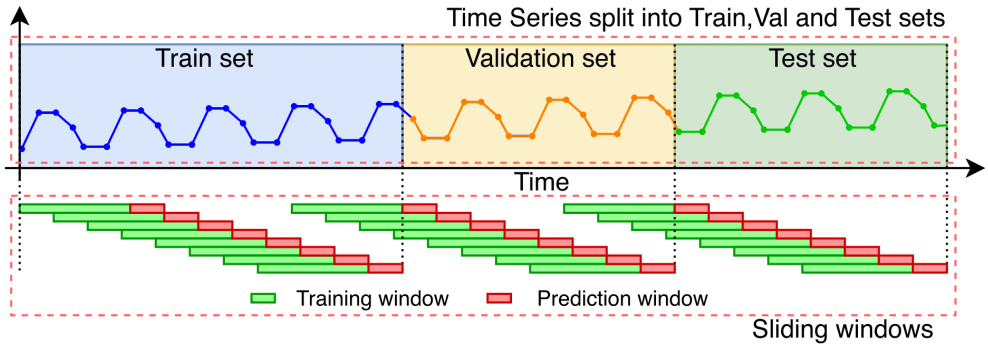


Figure 5.2 Non-overlapping sliding-window evaluation scheme [22].

2. At each window $w = 0, 1, \ldots, W-1$, the model receives the current context of length C, produces a forecast of H steps, and the context is then advanced by appending the H true test observations and discarding the oldest H values to maintain length C.

3. If the final window extends beyond the test set ($T \bmod H \neq 0$), only the first $T \bmod H$ predictions are retained.

This scheme simulates an operational deployment in which a model is periodically queried for a fixed-horizon forecast, always conditioned on the most recent available data [23]. All predictions are clipped to a minimum of zero after inference, because `n_bytes` cannot be negative. The same non-overlapping windowing scheme is also used to construct training examples for the deep learning models (GRU, LSTM, GRU-FCN). During per-series tuning (Section 5.3), the concatenation of the training and validation sets is segmented into input–target pairs with stride H, producing supervised samples that mirror the evaluation setting.

A *run* is one complete evaluation pass of a model over all benchmark series at a given resolution. GRU, LSTM, and GRU-FCN are trained from random initializations with stochastic mini-batch optimization, so a single pass may reflect a lucky or unlucky initialization. These models are therefore evaluated over 10 independent runs. For each series the per-run metric values are averaged, yielding a single value per (model, series) pair that is used in all subsequent aggregation and ranking. Deterministic models (Seasonal Naïve, SARIMA, Prophet, TimesFM, TTM, Chronos-2, Moirai) produce identical results on every run and are evaluated once.

5.3 Per-Series Tuning

The three deep learning models (GRU, LSTM, GRU-FCN) require a per-series tuning step before the sliding-window evaluation begins. This step encompasses input transformation, training-sample construction, and weight estima-

tion. Classical and foundation models require no per-series tuning—SARIMA and Prophet are refitted at every window, and the foundation models are applied zero-shot.

Log transform. As shown in Section 3.5.2, individual `n_bytes` series are heavily right-skewed. The transform $\tilde{x}_t = \log(1 + x_t)$ is therefore applied before normalization, following standard practice for positive-valued heavy-tailed data [3, 23]. The $\log(1 + x)$ variant handles zero-valued observations without requiring imputation.

Per-window instance normalization. After the log transform, each context window is independently normalized to $[0, 1]$ by its own minimum and maximum (per-window instance min–max scaling). Instance normalization, where each input is normalized by its own statistics rather than global ones, has been shown to mitigate distribution shift in time-series forecasting [51]. The same principle is applied here with min–max scaling instead of mean–standard-deviation standardization to preserve the non-negative structure of `n_bytes`.

5.4 Evaluation Measures

The benchmark spans three aggregation levels (IPs, subnets, institutions) whose typical byte counts differ by more than two orders of magnitude (Section 3.5.2), and two of the five benchmarks contain intermittent series. These properties impose two requirements on any evaluation metric: scale-independence, so that high-traffic institution series do not dominate the aggregate, and robustness to zero-valued observations. This section justifies the metric selection and describes the specific variant used for evaluation.

Excluded metrics. MAE and RMSE [23] are scale-dependent. When averaged across series whose scales differ by orders of magnitude, a small number of high-traffic series dominates the aggregate. MAE and RMSE are therefore not used for cross-series comparison.

SMAPE [34] assigns the maximum penalty of 200 whenever the actual is zero and the prediction is non-zero. Since the periodic spaces and workers benchmarks are dominated by zero-valued observations, SMAPE on these benchmarks measures the ability to output exact zeros rather than forecast quality.

R^2 is a goodness-of-fit statistic for regression that measures improvement over a constant-mean baseline. It is not designed for evaluating forecast accuracy [52]. Major forecasting benchmarks such as M5 and GIFT-Eval do not include R^2 in their metric sets [16, 14, 42].

MASE (primary metric). MASE normalizes each series by its own seasonal naïve error, making it scale-free and directly comparable across all benchmark series regardless of aggregation level [34]. It is the primary metric for evaluating point forecasts in GIFT-Eval [14] and fev-bench [42]. Following these benchmarks, the denominator is re-computed at each sliding window from the full history available at that point, so that the seasonal naïve baseline reflects the expanding context:

$$\text{MASE} = \frac{1}{W} \sum_{w=1}^{W} \frac{\frac{1}{H} \sum_{i=1}^{H} |y_{w,i} - \hat{y}_{w,i}|}{\frac{1}{N_w - m} \sum_{t=m+1}^{N_w} |y_{\text{hist},w,t} - y_{\text{hist},w,t-m}|} \tag{5.1}$$

where $y_{w,i}$ and $\hat{y}_{w,i}$ are the actual and predicted values at step i of window w, $y_{\text{hist},w}$ is the concatenation of training and validation sets with all test observations preceding window w, $N_w = |y_{\text{hist},w}|$ is its length, and m is the seasonal period: 24 for hourly data and 144 for 10-minute data (both representing daily seasonality).

Metric limitations. The MASE denominator is computed from historical data available at each window. When a series drifts between the training and test periods, this denominator no longer reflects the test-period scale. A traffic drop (high training traffic, low test traffic) produces a large denominator relative to small test-period errors, deflating MASE. Conversely, traffic growth inflates the errors relative to a small historical denominator.

5.5 Result Aggregation and Ranking

No single metric captures all aspects of model comparison. The five benchmarks differ in size (75 series for `seasonal` vs. 25 for `workers`), so naive pooling would let larger benchmarks dominate the ranking. Starting from the run-averaged per-series MASE values defined in Section 5.2, the values are aggregated per benchmark type by computing their mean and standard deviation. All rankings and aggregations are computed independently for each evaluation configuration (Table 5.1).

Average MASE rank (primary ranking). To prevent large benchmarks from dominating the comparison, a benchmark-balanced ranking is used as the primary verdict:

1. **Per-series rank:** models are ranked by their (run-averaged) MASE on each series (ascending; ties receive the average of tied ranks).

2. **Per-benchmark average rank:** per-series ranks are averaged within each benchmark type.
3. **Overall average rank:** the per-benchmark average ranks are averaged uniformly:

$$\bar{r}_m = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \bar{r}_{m,b} \tag{5.2}$$

where $\bar{r}_{m,b}$ is the average rank of model m within benchmark b and $\mathcal{B}$ is the set of benchmark types.

A lower average rank is better. Rank 1 means the model was consistently best across benchmarks.

Skill score (supplementary ranking). The average rank captures ordinal performance but not the magnitude of improvement over a baseline. As a complementary view, the marginal skill score proposed by Shchur et al. [42] quantifies how much a model reduces forecasting error compared to a fixed baseline β (Seasonal Naïve) on average:

$$S_j = 1 - \sqrt[R]{\prod_{r=1}^{R} \mathrm{clip}\left(\frac{E_{rj}}{E_{r\beta}};\ \ell,\ u\right)} \tag{5.3}$$

where E_{rj} is the MASE of model j on series r, $E_{r\beta}$ is the baseline MASE, R is the number of series, and $\mathrm{clip}(x; \ell, u)$ restricts the ratio to $[10^{-2}, 100]$. S_j ranges from 1 (perfect forecasts) to $-\infty$. Positive values indicate improvement over the baseline.

Pairwise win rate. A pairwise win rate matrix is computed following [42]: for each ordered pair of models (m_A, m_B), the win rate is the fraction of series on which m_A achieves a lower MASE than m_B, with ties counted as half a win for each model:

$$W(m_A, m_B) = \frac{1}{|\mathcal{S}_{A,B}|} \sum_{s \in \mathcal{S}_{A,B}} \big(\mathbb{1}(\mathrm{MASE}_{m_A}(s) < \mathrm{MASE}_{m_B}(s)) + 0.5 \cdot \mathbb{1}(\mathrm{MASE}_{m_A}(s) = \mathrm{MASE}_{m_B}(s))\big) \tag{5.4}$$

where $\mathcal{S}_{A,B}$ is the set of series for which both models produce a result. The resulting $M \times M$ matrix reveals head-to-head dominance patterns that a single aggregate number may obscure.

Pairwise skill score. The pairwise skill score o [42] complements the win rate by quantifying the magnitude of error reduction between two models. For models m_A and m_B:

$$S(m_A, m_B) = 1 - \sqrt[R]{\prod_{r=1}^{R} \mathrm{clip}\left(\frac{E_{r,m_A}}{E_{r,m_B}};\ \ell,\ u\right)} \tag{5.5}$$

where $E_{r,m}$ is the MASE of model m on series r, $R = |\mathcal{S}_{A,B}|$, and $\mathrm{clip}(x; \ell, u)$ restricts the ratio to $[10^{-2}, 100]$. A positive score indicates that m_A reduces error relative to m_B on average and the win rate and pairwise skill score together distinguish whether a model wins frequently (win rate) and by how much (skill score).

Statistical significance testing. Rank differences are tested with the Friedman test followed by the Nemenyi post-hoc procedure [53], run separately per benchmark within each configuration. The five Friedman p-values are corrected with the Holm–Bonferroni method [54], and this correction applies only to the Friedman tests. Pairwise model comparisons rely on the Nemenyi critical-difference statistic, which controls the family-wise error rate across model pairs internally. Results are presented as Critical Difference (CD) diagrams, where models connected by a horizontal bar are not significantly different at $\alpha = 0.05$.

Chapter 6

Framework

This chapter describes NetTS-eval, a Python framework for automated evaluation of time series forecasting models on the benchmarks defined in Chapter 4. The framework handles benchmark data management, model abstraction, sliding-window evaluation, metric computation, result persistence, and post-hoc analysis with visualization. It is designed to be extensible: adding a new model requires implementing a single abstract interface. The data layer is built on *pandas* [55] and *numpy* [56]. All deep learning and foundation model inference uses *PyTorch* [57] as the common backend.

Package Structure

The framework is organized as a Python package *netts-eval* with three subpackages and two top-level entry-point scripts:

```
netts-eval/
├── models/ ..................... forecasting model wrappers (10 models)
├── benchmarks/ ..................... benchmark loading and data access
├── evaluation/ ..................... evaluator, metrics, result storage
└── utils/ ..................... EvaluationConfig dataclass
run_evaluation.py ..... CLI entry point for running evaluations
save_benchmarks.py ..... data export from cesnet-tszoo to CSV/Parquet
result_exploration/
├── analysis.py ..................... result loading, ranking, aggregation
├── plots.py ..... visualization functions
└── generate.py ..... batch generation of all tables and plots
metacentrum/ ..... PBS job submission for MetaCentrum HPC
```

6.1 Benchmark Data Management

Data Export

Benchmark data are exported once and stored on disk rather than loaded from *cesnet-tszoo* at evaluation time. This makes the evaluation self-contained — it does not depend on the upstream library version or network access — and the exported directory structure groups series by benchmark type and aggregation level for easy inspection. The export pipeline `save_benchmarks.py` extracts benchmark time series from the CESNET-TimeSeries24 dataset via the *cesnet-tszoo* library [48]. For each of the five benchmark types, the script reads pre-curated series identifiers from CSV files containing `(level, ts_id)` pairs, where `level` is one of Institutions, Subnets, or IPs. Each level is mapped to a *cesnet-tszoo* source type at the requested temporal resolution (hourly or 10-minute); the selected series are loaded with the `n_bytes` feature, zero-filled missing values, and datetime timestamps. The data are saved in both long-format CSV and Apache Parquet, together with a `metadata.json` file recording the train/validation split boundary indices and per-source series identifiers.

Runtime Loading

At evaluation time, benchmark data are loaded from the exported files rather than from the raw dataset. The `BenchmarkLoader.load()` method reads `metadata.json` to discover available aggregation levels and their split boundaries, then loads each source CSV into a `BenchmarkSource` dataclass. This dataclass holds the training, validation, and test DataFrames for one aggregation level of one benchmark type, produced by slicing each series group at the boundary indices from the metadata. Series are accessed individually via the `get_series(ts_id, split)` method, which filters the appropriate DataFrame by the series identifier column.

6.2 Model Abstraction

Base Interface

All forecasting models inherit from the abstract base class `BaseModel`, which defines the following contract:

The interface supports two inference paths. In the *sequential path* (default), the evaluator calls `fit()` with the context DataFrame for each sliding window, followed by `predict()` to produce the forecast. This path is used by Seasonal Naïve, SARIMA, Prophet, TTM, and the deep learning models. In the *batch path* (`supports_batch = True`), the evaluator constructs all context windows upfront and passes them to `predict_batch()` in a single call. This path is

```python
class BaseModel(ABC):
    supports_batch: bool = False

    def tune(self, train_data, val_data,
             target_column, ts_id) -> dict | None
    def get_required_context_size(self) -> int | None
    def fit(self, train_data, target_column) -> None
    def predict(self, horizon) -> pd.Series
    def predict_batch(self, contexts,
                      target_column, horizon) -> list[pd.Series]
    def get_params(self) -> dict
    def get_name(self) -> str
```

Code listing 1 Model interface.

used by TimesFM, Chronos-2, and Moirai, whose architectures support parallel inference over multiple contexts.

The optional `tune()` method is invoked once per series before the sliding-window loop. It receives the full training and validation splits and is used for per-series model training (GRU, LSTM, GRU-FCN), scaler fitting, or hyperparameter tuning. The optional `get_required_context_size()` allows a model to override the user-specified look-back with a fixed context length; TTM uses this to read its required context from the model configuration, and the deep learning models return their `lookback` parameter.

Model Wrappers

Each of the ten evaluated models (Section 5.1) is implemented as a `BaseModel` subclass in a separate module under `netts-eval/models/`. Table 6.1 lists the underlying library and model size for each wrapper. The evaluation-relevant configuration (hyperparameters, preprocessing, normalization) is described in Chapter 5; the wrappers handle only input formatting, library-specific API calls, and error recovery. The only non-trivial error-recovery logic is in the SARIMA wrapper: if the default fit raises a `LinAlgError` (singular Hessian), a fallback fit with relaxed stationarity and invertibility constraints is attempted. The deep learning models are implemented using the *tsai* library [58], chosen to replicate the setup of Koumar et al. [22].

Table 6.1 Libraries and model sizes for each model wrapper.

Model	Library	Parameters
Seasonal Naïve	built-in	—
SARIMA	*statsmodels* SARIMAX	—
Prophet	*prophet* [30]	—
TimesFM 2.5	*timesfm* [8]	200 M
TTM	*granite-tsfm* [47]	1 M
Chronos-2	*chronos-forecasting* [9]	120 M
Moirai 2.0-R-Small	*uni2ts* [10]	11.4 M
GRU, LSTM, GRU-FCN	*tsai* [58]	—

6.3 Evaluation Engine

The `EvaluationConfig` dataclass holds all evaluation parameters: the forecast horizon, context size, target column (default `n_bytes`), list of metric names, data frequency, and the seasonal period m for the scaled metrics (set automatically: $m = 24$ for hourly data, $m = 144$ for 10-minute data).

Evaluator

The `Evaluator` class orchestrates the sliding-window evaluation described in Section 5.2 for one model over one `BenchmarkSource`. For each series in the source, the evaluator calls `model.tune()` once (recording the time as seconds per 100 data points), precomputes all context windows, dispatches inference through the batch or sequential path depending on the model's `supports_batch` flag, and computes per-series metrics from the collected predictions and true values. Inference time is recorded separately for each series. The result is a typed dictionary containing the series identifier, number of windows, predictions, true values, timestamps, metric values, optional tuned parameters, and timing information.

Metric Computation and Results

Metrics are implemented with a registry pattern: each metric is a function registered in a dictionary, and the `calculate_metrics()` dispatcher invokes the appropriate function by name. Adding a new metric requires only registering a function. MASE uses the windowed computation described in Section 5.4.

The primary output is per-series metrics, the lowest granularity at which results are meaningful. All ranking and aggregation methods in Section 5.5 operate on these values. Per-timestep predictions are optionally saved alongside, allowing metrics to be recomputed after the fact if the metric set changes. Per-run and cross-run summaries are pre-aggregated for quick inspection with-

out running the analysis pipeline. The output directory structure is shown in Section 6.4.

6.4 Command-Line Interface

The entry point `run_evaluation.py` accepts arguments for model selection, benchmark types, context/horizon configuration, and number of runs. A factory function instantiates the requested model class and applies model-specific defaults (e.g. Czech holiday calendar for Prophet, log transform for the deep learning models). The execution loop iterates over runs, benchmarks, and aggregation levels, calling the evaluator and saving results after each source. Each evaluation produces a timestamped output directory:

```
results/{timestamp}_{model}/
├── config.json ........................ run metadata and parameters
├── model_config.json ..................model hyperparameters
├── summary.csv ........................cross-run aggregate
└── run_1/
    ├── {bench_type}_metrics.csv
    ├── {bench_type}_predictions.csv ...................optional
    └── summary.csv
```

6.5 Result Analysis and HPC Deployment

The *result_exploration* package provides post-hoc analysis of evaluation outputs. A central configuration dictionary maps experiment names (e.g. `168_24`, `744_168`) to per-model result directory paths. The `load_all_results()` function reads per-series metric CSVs, pools results across runs, and returns a single DataFrame indexed by series identifier, aggregation level, model, and benchmark type. Missing metrics are recomputed on-the-fly from stored predictions when available. The ranking and aggregation functions implement the methods defined in Section 5.5, and a visualization module built on *matplotlib* [59], *seaborn* [60], and *plotly* [61] generates all plots used in Chapter 7.

The deep learning models (GRU, LSTM, GRU-FCN) are trained individually per series, and each is evaluated over 10 independent runs to account for stochastic initialization (Section 5.2). With 532 series, four evaluation configurations, and three model architectures, this amounts to thousands of individual training passes — making local execution impractical. The framework therefore includes integration with MetaCentrum, the Czech national HPC infrastructure. A submission script classifies models as GPU-bound or CPU-bound, assigns appropriate resource requests, and submits one PBS job per model. Model name, benchmark types, and evaluation parameters are passed as environment variables to the job script, which activates the runtime environment and invokes `run_evaluation.py`.

Chapter 7

Experiments and Results

This chapter presents the results of evaluating the ten forecasting models described in Section 5.1 across the five benchmarks from Chapter 4. The evaluation follows the protocol, metrics, and aggregation procedures defined in Chapter 5. All results are reported using MASE and the benchmark-balanced average rank (Section 5.5) as the primary ranking. Results are organized from overall model ranking through per-benchmark analyses.

7.1 Overall Model Ranking

Table 7.1 presents the average MASE rank for each model per benchmark, pooled across all four evaluation configurations (Table 5.1).

Table 7.1 Average MASE rank per benchmark, pooled across all four configurations. Lower is better. Models are ordered by their average rank. Bold marks the best model per column.

Model	Drift	Season.	Per. Sp.	Work.	No Temp.	Avg
TimesFM	2.67	2.68	**3.14**	3.50	**2.77**	**2.95**
Chronos-2	**2.15**	**1.88**	4.42	4.58	3.02	3.21
Moirai	3.39	4.22	4.63	4.18	3.21	3.93
GRU	5.75	6.32	4.72	**3.03**	4.76	4.91
GRU-FCN	5.78	5.17	4.84	4.05	5.48	5.06
LSTM	6.44	6.54	4.95	3.44	5.53	5.38
TTM	5.49	4.96	6.44	6.48	5.61	5.80
SARIMA	7.42	7.24	6.91	8.27	7.82	7.53
S. Naïve	7.82	7.86	7.20	8.14	8.71	7.94
Prophet	8.10	8.13	7.75	9.33	8.10	8.28

Table 7.2 quantifies the magnitude of improvement over the Seasonal Naïve baseline using the skill score (Equation 5.3), expressed as a percentage of error reduction.

Table 7.2 Skill score (%) relative to Seasonal Naïve per benchmark, pooled across all four configurations. Positive values indicate improvement over the baseline. Bold marks the best model per column.

Model	Drift	Season.	Per. Sp.	Work.	No Temp.	Avg
TimesFM	28.3	26.2	**42.3**	44.2	**36.8**	**35.5**
Chronos-2	**29.4**	**29.0**	−35.7	43.4	36.4	20.5
Moirai	27.4	22.3	−33.7	**44.3**	36.6	19.4
TTM	11.5	15.7	−76.1	16.4	20.8	−2.3
SARIMA	−0.7	0.8	−2.8	−8.1	−2.7	−2.7
GRU-FCN	−0.4	−0.5	−63.1	35.5	10.2	−3.6
Prophet	−6.6	−4.4	−14.4	−21.6	1.2	−9.2
GRU	10.3	−0.7	−333.6	27.3	25.4	−54.3
LSTM	5.0	1.3	−389.9	27.5	25.4	−66.1

Foundation models consistently occupy the top three positions. TimesFM achieves the best average rank (2.95) and the highest average skill (+35.5%), leading on Periodic Spaces and No Temporal Structure. Chronos-2 ranks first on Drift and Seasonality (ranks 2.15 and 1.88, skill +29.4% and +29.0%). Moirai completes the foundation model tier with an average rank of 3.93 and positive skill on all benchmarks except Periodic Spaces.

The per-series deep learning models (GRU, LSTM, GRU-FCN) occupy the middle tier (average ranks 4.91–5.38). Their performance varies strongly by benchmark type: GRU achieves the best rank on Workstation Activity (3.03), while all three rank 5th–7th on the low-sparsity benchmarks (Drift, Seasonality). Their average skill is reduced below zero by Periodic Spaces (−333.6% for GRU, −389.9% for LSTM), masking moderate positive skill on the remaining benchmarks. TTM sits between the foundation models and deep learning on Seasonality (rank 4.96, skill +15.7%) but degrades on the sparse benchmarks.

Statistical models (SARIMA, Prophet) and Seasonal Naïve cluster at the bottom (average ranks 7.53–8.28) with near-zero or negative skill across all benchmarks.

The ranking also varies across aggregation levels (institutions, subnets, IPs). Figure 7.1 shows the mean MASE per model and aggregation level, pooled across all configurations. The easiest level is not the same for all model families: foundation and recurrent models forecast IP-level series most accurately and subnets least accurately, while SARIMA and GRU-FCN invert this ordering. Within a single model, the spread across levels can be large enough to mask a complete failure at one level behind an acceptable aggregate score.

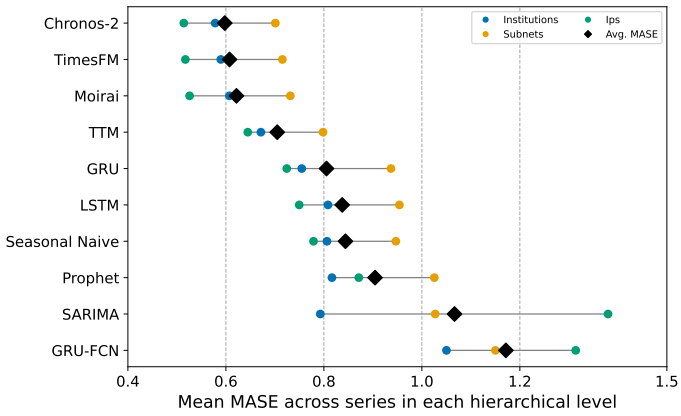


Figure 7.1 Mean MASE per model and aggregation level, pooled across all four configurations and the three benchmarks with all three levels (Drift, Seasonality, No Temporal Structure). Diamond marks the mean across levels.

7.2 Statistical Significance

The Friedman test with the Nemenyi post-hoc procedure [53] is applied per benchmark within each of the four configurations (Table 5.1). All 20 tests reject the null hypothesis of equal model ranks at $\alpha = 0.05$ ($p < 10^{-15}$ after Holm–Bonferroni correction). The per-test χ^2 statistics are reported in Table D.1 in Appendix D.

The resulting CD diagrams show how the significance groupings shift across benchmark types. Two representative cases are shown below and the complete set of 20 diagrams is provided in Appendix D.

Figure 7.2 shows the Data Drift benchmark under the long hourly configuration (h/744/168, $n = 75$). The three foundation models form a group at ranks 2.29–2.47, separated from all remaining models. Seasonal Naïve (rank 5.33) occupies an intermediate position, and all three deep learning models cluster at ranks 7.19–7.97, statistically indistinguishable from Prophet and worse than the trivial baseline.

Figure 7.3 shows the Workstation Activity benchmark under the long 10-minute configuration (m/1008/144, $n = 25$). The ranking inverts. GRU (1.60) and LSTM (1.72) occupy the top positions, significantly ahead of all foundation models. TimesFM (4.48) and Chronos-2 (4.60) drop to the middle of the ranking. The same model families that rank last on Data Drift rank first on Workstation Activity.

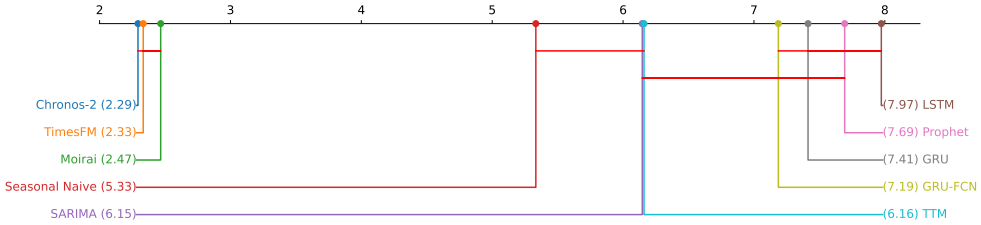


Figure 7.2 CD diagram from the Nemenyi post-hoc test on the Data Drift benchmark (h/744/168, $n = 75$). Connected models are not significantly different at $\alpha = 0.05$.

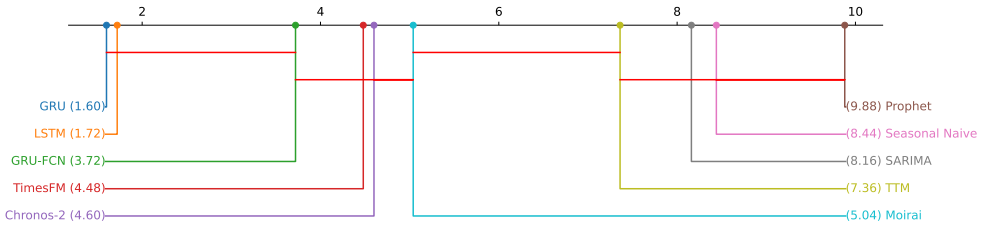


Figure 7.3 CD diagram from the Nemenyi post-hoc test on the Workstation Activity benchmark (m/1008/144, $n = 25$). Connected models are not significantly different at $\alpha = 0.05$.

7.3 Pairwise Comparisons

Figure 7.4 shows the pairwise win rates pooled across all four configurations. Among the foundation models, Chronos-2 achieves a lower MASE than TimesFM on 57% of individual series, despite having a slightly worse average rank. TimesFM outperforms Moirai on 78% of series, making this the clearest pairwise separation within the foundation model tier. All three foundation models outperform Seasonal Naïve on 94–98% of series. The deep learning models win against Seasonal Naïve on only 66–70% of series (pooled), consistent with their negative skill in multiple configurations.

Pairwise skill scores. The pairwise skill score (Section 5.5) complements the win rate by quantifying the magnitude of error reduction. Figure 7.5 shows the pooled pairwise skill scores. A positive score indicates that the row model reduces error relative to the column model on average. Across configurations and benchmarks, the three foundation models show consistently positive pairwise skill against all other model categories. Within the foundation model tier, TimesFM achieves positive pairwise skill against Moirai on all benchmarks but near-zero or slightly negative pairwise skill against Chronos-2 on the Seasonality and Data Drift benchmarks, confirming the pattern observed in the win rates: Chronos-2 and TimesFM trade leadership depending on the benchmark type.

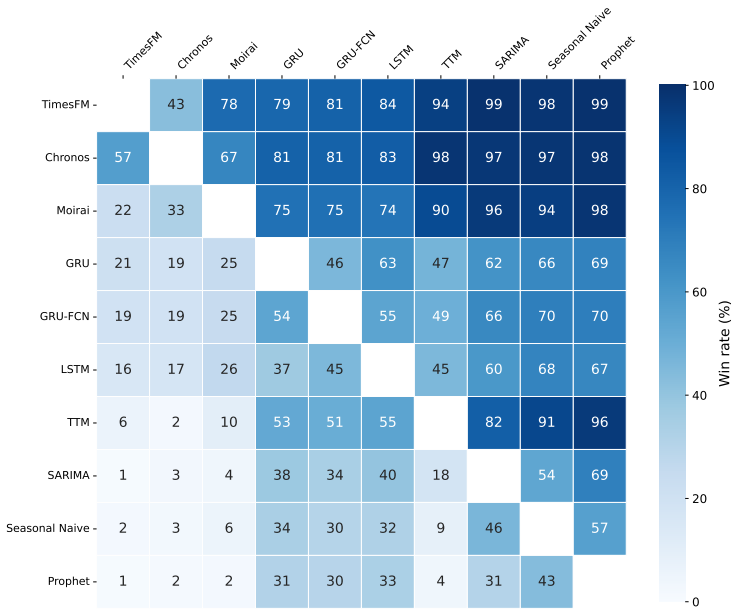


Figure 7.4 Pairwise per-series win rate (%). The cell at row A, column B reports the percentage of series on which model A achieves strictly lower MASE than model B, pooled across all four configurations. Darker blue indicates a higher win rate for the row model. The diagonal is undefined.

Figure 7.5 Pairwise skill score (%). The cell at row A, column B reports the percentage of error reduction that model A achieves relative to model B, pooled across all four configurations. Positive values (green) indicate that the row model outperforms the column model, negative values (red) indicate the opposite. The diagonal is undefined.

7.4 Sensitivity to Configuration

The results presented so far pool all four evaluation configurations (Table 5.1). This section disaggregates by configuration to examine how context length, forecast horizon, and temporal resolution affect model performance.

Table 7.3 shows the average rank per configuration, and Table 7.4 shows the corresponding skill scores. The two tables reveal three distinct sensitivity patterns.

Table 7.3 Average MASE rank per configuration, averaged across all five benchmarks. Lower is better. Bold marks the best model per column.

Model	h/168/24	h/744/168	m/288/144	m/1008/144
TimesFM	**3.19**	**2.79**	**2.62**	3.21
Chronos-2	3.65	3.17	2.96	**3.05**
Moirai	4.19	3.21	3.85	4.45
GRU	4.31	5.67	5.77	3.91
GRU-FCN	5.29	5.39	4.92	4.65
LSTM	4.80	6.29	5.88	4.54
TTM	3.85	6.73	5.98	6.62
SARIMA	9.06	6.61	7.05	7.42
S. Naïve	8.13	6.94	8.23	8.47
Prophet	8.53	8.20	7.74	8.68

Table 7.4 Skill score (%) per configuration, averaged across all five benchmarks. Positive values indicate improvement over the Seasonal Naïve baseline. Bold marks the best model per column.

Model	h/168/24	h/744/168	m/288/144	m/1008/144
TimesFM	**36.0**	**32.2**	**36.3**	**37.7**
Chronos-2	26.6	21.0	16.3	18.1
Moirai	25.2	21.3	14.8	16.1
TTM	35.0	−24.0	−5.9	−14.5
SARIMA	−25.8	−0.8	7.9	7.9
GRU-FCN	10.9	−26.7	−8.0	9.3
Prophet	−9.9	−20.0	1.3	−8.1
GRU	20.5	−8.6	−242.5	13.6
LSTM	20.0	−10.6	−285.8	12.0

Foundation models are configuration-stable. TimesFM maintains a skill score between +32% and +38% across all four configurations and never drops below rank 3.21. Chronos-2 and Moirai show a moderate decrease from

the hourly short configuration (+26.6%/+25.2%) to the 10-minute configurations (+16–18%/+15–16%), but remain positive throughout. The three foundation models occupy the top three ranks in three of four configurations. The only exception is the long 10-minute configuration (m/1008/144), where GRU enters the top three at rank 3.91.

The per-series deep learning models exhibit the strongest configuration dependence. On h/168/24, GRU and LSTM achieve positive skill (+20.5% and +20.0%), but moving to the longer horizon (h/744/168) turns their skill negative. The effect is most extreme on m/288/144, where only 2 days of context are available: GRU skill drops to −242.5% and LSTM to −285.8%, driven by failures on the sparse benchmarks. With longer 10-minute context (m/1008/144, 7 days), both models recover to positive skill (+13.6% and +12.0%), confirming that the failures stem from insufficient context rather than an intrinsic modeling limitation.

TTM achieves rank 3.85 and skill +35.0% on the hourly short configuration, comparable to TimesFM. On all other configurations, its rank drops to 5.98–6.73 and its skill is negative. Each configuration requires a different pretrained checkpoint with a fixed context length and horizon [47]. The h/168/24 checkpoint (revision r2.1) was pretrained on ≈1 B samples from 30 datasets with MAE loss, while the remaining three use r2 checkpoints pretrained on ≈700 M samples from 14 datasets with MSE loss [62]. The combination of more training data and a loss function aligned with the MAE-based MASE metric gives the h/168/24 checkpoint an advantage.

7.5 Summary

The five benchmarks yield systematically different scores and rankings, confirming that no single aggregate leaderboard faithfully represents model behavior across all traffic patterns.

Foundation models dominate the overall ranking but not uniformly. TimesFM, Chronos-2, and Moirai occupy the top three average ranks and reduce error against the Seasonal Naïve baseline by 19–36% on average. They maintain positive skill across all four evaluation configurations. However, on Workstation Activity with sufficient context, GRU and LSTM surpass all foundation models, achieving ranks 1.60 and 1.72 on the long 10-minute configuration.

Per-benchmark views expose model-specific blind spots. Periodic Spaces is the strongest discriminator: every model except TimesFM posts negative average skill. The recurrent models achieve the best per-series ranks on Workstation Activity, a behavior that other benchmarks mask entirely. Chronos-2 leads on Seasonality and Data Drift but degrades on Periodic Spaces.

Configuration sensitivity separates the model families further. Foundation models are stable across all four configurations (TimesFM skill 32–38%). Deep learning models collapse when context is insufficient (m/288/144) but recover

with longer context (m/1008/144). TTM is competitive only on the single configuration that uses an L1-trained checkpoint with a larger pretraining corpus.

Foundation models complete a full series evaluation in under 0.5 s across all configurations. Deep learning models take 0.2–2.9 s due to the per-series training step. SARIMA is the slowest at 15–41 s per series because it refits the full SARIMAX model on every sliding window. Per-configuration wall-clock times are reported in Table E.1 in Appendix E.

Chapter 8
Discussion

Two prior studies evaluated forecasting models on CESNET-TimeSeries24. Koumar et al. [22] found that recurrent deep learning models outperform statistical baselines across all three aggregation levels. The present evaluation confirms this at the aggregate level but shows that the advantage varies across benchmarks (Section 7.5): on Workstation Activity the deep learning models rank among the best, while on Drift and Seasonality they fall below the Seasonal Naïve baseline in several configurations. Liu et al. [11] showed that zero-shot TTM outperforms per-series GRU and LSTM at institution and subnet levels. The results here show that TTM's competitiveness depends on which checkpoint is used, matching the other foundation models only under the hourly short configuration (Section 7.4).

The variation in model rankings across benchmarks is consistent with the broader critique of aggregate leaderboards in multi-domain evaluation suites such as GIFT-Eval [14] and fev-bench [42].

This study has several limitations. Although threshold sensitivity sweeps confirm that the qualifying pools vary smoothly and no chosen cutoff sits on a knife-edge (Appendix C), the effect of threshold changes on the final model rankings was not evaluated. All benchmarks target a single feature (`n_bytes`), and the conclusions may not generalize to other traffic metrics or multivariate settings. The MASE denominator, computed from historical data, introduces a known bias on drifting series (Section 5.4). All foundation models were evaluated zero-shot only, so the results do not reflect what domain-specific fine-tuning could achieve. The supervised deep learning models are limited to recurrent architectures. Including trained transformer-based models would provide a more current supervised baseline. Two of the five benchmarks (Periodic Spaces and Workstation Activity) are dominated by zero-valued observations, yet no intermittent-demand method was evaluated. Dedicated approaches such as Croston's method [63], the Syntetos–Boylan Approximation [64], and the Teunter–Syntetos–Babai method [65] decompose demand into occurrence and size components and are designed precisely for this regime. Evaluating them

alongside zero-inflated post-processing on top of foundation models would test whether the poor performance observed on these benchmarks stems from the intermittent structure itself or from the models' general forecasting ability.

With current knowledge, several design choices deserve reconsideration. The benchmark selection thresholds were set by domain knowledge and visual inspection of score distributions. Replacing them with data-driven cutoffs (e.g., clustering or quantile-based selection) and formal criteria for what qualifies a series for each benchmark could reduce subjectivity in the construction process. The deep learning models were trained individually per series with fixed hyperparameters from Koumar et al. [22]. A broader hyperparameter search and a global training regime pooling series within each benchmark could clarify whether the observed failures stem from configuration, data scarcity, or architecture.

Chapter 9
Conclusion

This thesis set out to design property-specific diagnostic benchmarks for network traffic forecasting, select state-of-the-art models from multiple forecasting paradigms, implement an evaluation framework, and compare model performance.

Five diagnostic benchmarks were constructed from the CESNET-TimeSeries24 dataset at two temporal resolutions, each isolating a single structural characteristic through predefined selection criteria applied across three aggregation levels. Ten forecasting models spanning classical statistical, per-series deep learning, and zero-shot foundation model paradigms were selected and configured. An extensible evaluation framework, NetTS-eval, was implemented to automate sliding-window evaluation, metric computation, and result analysis. The experimental evaluation confirmed that the five benchmarks yield systematically different model rankings. Foundation models achieved the best overall ranks and reduced forecasting error over the baseline on most benchmarks, but recurrent deep learning models outperformed them on workstation activity patterns. These differences, discussed in Chapter 8, would be invisible under a single aggregate ranking and confirm the value of property-specific evaluation. All goals were therefore achieved.

Several directions for future work remain open. The evaluation could be extended to multivariate forecasting with additional traffic features. The benchmark suite could incorporate further traffic characteristics such as burstiness or long-range dependence, or be applied to other network datasets. Fine-tuning foundation models, evaluating global deep learning training regimes, and validating that threshold choices do not alter model rankings would address the limitations identified in Chapter 8.

Appendix A

`n_bytes` Properties at 10-minute Resolution

Tables A.1 and A.2 report the 10-minute counterparts of Tables 3.3 and 3.4. The same qualitative patterns hold, amplified by the finer granularity: sparsity increases and raw skewness and kurtosis roughly triple at every level.

Table A.1 Percentage of non-zero datapoints at the 10-minute resolution (mean and median across all series at each aggregation level).

Level	Mean (%)	Median (%)
Institutions	91.6	99.9
Subnets	81.3	99.4
IP addresses	17.0	5.5

Table A.2 Median of per-series distribution statistics at each aggregation level at the 10-minute resolution, computed on raw `n_bytes` and on log-transformed values $\tilde{x}_t = \log(1 + x_t)$.

Level	Transform	Skewness	Kurtosis	Max/Median
Institutions	raw	15.0	429.5	766.5
	transformed	−0.4	1.5	1.4
Subnets	raw	19.4	710.5	1041.2
	transformed	−0.4	1.9	1.5
IPs	raw	58.6	4448.0	3483.8
	transformed	4.4	19.2	2.1

Appendix B

Selected Benchmark Series

This appendix lists the series IDs selected for each benchmark at both the hourly and 10-minute resolutions. Each ID corresponds to the series ID in the original CESNET-TimeSeries24 dataset [17]. The selection methodology is described in Chapter 4. All 10-minute parameters are scaled by a factor of 6 from their hourly values. Tables B.1–B.5 report the candidate counts at each filtering stage for the 10-minute resolution (the hourly counts are given in Chapter 4).

Table B.1 Seasonality benchmark: candidate counts at each filtering stage (10-minute).

Level	Total	After sparsity filter	Qualifying pool ($\geq$0.7)
Institutions	283	266	63
Subnets	548	468	71
IPs	275 124	34 729	548

Table B.2 Data Drift benchmark: candidate counts at each filtering stage (10-minute).

Level	Total	After sparsity filter	Has drift	Qualifying pool
Institutions	283	235	—	91
Subnets	548	373	—	166
IPs	1 000	47	—	25

Table B.3 Periodic Spaces benchmark: candidate counts at each filtering stage (10-minute).

Level	Total	After sparsity band	Qualifying pool ($\geq$0.6)
IPs (sample)	1 000	471	9
IPs (full)	275 124	133 780	1 580

Table B.4 Workstation Activity benchmark: candidate counts at each filtering stage (10-minute).

Level	Total	After sparsity filter	Qualifying pool ($\geq$0.6)
IPs (sample)	1 000	203	0
IPs (full)	275 124	49 513	34

Table B.5 No Temporal Structure benchmark: candidate counts at each filtering stage (10-minute).

Level	Total	After sparsity filter	Qualifying pool ($\leq$0.1)
Institutions	283	266	19
Subnets	548	468	58
IPs	1 000	150	50

Table B.6 Selected series for the 10-minute aggregation interval and each benchmark. †No Temporal Structure – Institutions: all 19 qualifying series selected (pool size < 25).

Benchmark	Level	Series IDs
Seasonality	Inst.	0, 7, 9, 10, 12, 15, 23, 39, 41, 48, 50, 53, 63, 95, 100, 102, 127, 147, 161, 175, 186, 206, 216, 228, 240
	Sub.	0, 17, 91, 130, 136, 141, 148, 181, 184, 200, 241, 244, 250, 253, 261, 264, 315, 318, 321, 350, 381, 397, 438, 440, 445
	IPs	83, 2938, 3052, 3135, 4729, 4897, 15032, 17447, 17669, 18214, 26685, 36509, 38586, 40138, 53602, 77052, 78489, 99092, 122315, 186070, 204405, 349693, 435318, 747744, 1544688
Data Drift	Inst.	4, 11, 26, 38, 53, 61, 69, 77, 89, 92, 103, 107, 123, 127, 141, 154, 164, 186, 195, 200, 202, 211, 231, 254, 259
	Sub.	33, 44, 50, 51, 58, 59, 67, 80, 86, 117, 142, 154, 189, 248, 250, 262, 310, 316, 319, 326, 341, 403, 408, 425, 517
	IPs	120, 122, 1367, 1368, 1580, 1605, 1774, 3271, 10256, 10396, 11254, 41990, 44252, 75195, 78471, 109544, 121596, 171452, 195100, 242764, 259860, 565727, 605036, 617662, 745827
Periodic Spaces	IPs	10114, 10347, 19798, 105731, 137398, 210681, 219080, 223496, 224079, 230203, 288065, 292005, 293673, 324745, 355858, 377858, 479727, 620344, 630736, 699221, 766565, 849573, 1861858, 1933649, 2003176
Workstation Activity	IPs	3846, 21939, 29898, 53673, 111517, 187291, 272399, 303078, 323771, 330940, 333372, 351284, 367407, 438509, 460989, 508147, 514692, 517910, 568739, 672019, 673645, 811277, 818999, 1488481, 1513237
No Temporal Structure†	Inst.	19, 20, 37, 83, 84, 92, 121, 163, 172, 225, 237, 254, 255, 259, 266, 270, 272, 275, 281
	Sub.	2, 19, 26, 40, 41, 49, 77, 80, 113, 119, 138, 205, 269, 270, 288, 297, 300, 392, 404, 429, 480, 494, 514, 519, 535
	IPs	1845, 3271, 10196, 20342, 41990, 67484, 78471, 109544, 119651, 125740, 151737, 290222, 293838, 314181, 360850, 363446, 379493, 405441, 564287, 599190, 617379, 781973, 792059, 797312, 1604957

Table B.7 Selected series for the 1-hour aggregation interval and each benchmark. †No Temporal Structure – Institutions: all 13 qualifying series selected (pool size < 25).

Benchmark	Level	Series IDs
Seasonality	Inst.	15, 16, 38, 39, 52, 56, 63, 70, 96, 112, 123, 132, 147, 158, 183, 194, 195, 204, 208, 217, 231, 235, 249, 252, 261
	Sub.	91, 133, 136, 145, 146, 152, 179, 197, 214, 261, 268, 274, 315, 381, 388, 391, 395, 405, 408, 451, 452, 494, 509, 517, 530
	IPs	1370, 1749, 1784, 3313, 6005, 6768, 10280, 13911, 35182, 36419, 38213, 61048, 114939, 119674, 132516, 137574, 139017, 194323, 250872, 305866, 347314, 472452, 518935, 626876, 777653
Data Drift	Inst.	4, 9, 20, 26, 56, 63, 70, 89, 90, 99, 118, 127, 132, 137, 144, 154, 172, 182, 185, 209, 213, 238, 251, 261, 269
	Sub.	26, 43, 48, 96, 133, 143, 149, 171, 241, 264, 280, 353, 363, 373, 408, 419, 444, 457, 460, 475, 477, 498, 501, 523, 533
	IPs	1367, 1368, 1605, 1774, 3271, 10396, 11254, 41990, 43611, 44252, 75195, 121596, 125740, 132509, 149504, 195100, 242764, 300547, 322201, 512788, 565727, 617662, 745827, 756220, 783215
Periodic Spaces	IPs	101, 1385, 1845, 5101, 11204, 11212, 35748, 121748, 162890, 165515, 232476, 237694, 302806, 307400, 334600, 355332, 430173, 566752, 594145, 720439, 722293, 737244, 1512833, 1536107, 1563916
Workstation Activity	IPs	3846, 21943, 61466, 71574, 142157, 187291, 222483, 258691, 286014, 288481, 305252, 308900, 328623, 365864, 384820, 411626, 422139, 460989, 514692, 582575, 591569, 598086, 733725, 754726, 790615
No Temporal Structure†	Inst.	37, 83, 237, 256, 259, 264, 267, 270, 272, 277, 279, 280, 281
	Sub.	23, 26, 41, 45, 46, 52, 57, 73, 76, 80, 89, 138, 185, 199, 300, 364, 404, 430, 461, 516, 522, 532, 535, 537, 542
	IPs	10158, 10196, 39965, 119651, 125740, 151737, 156921, 170487, 195100, 215374, 290222, 330012, 363446, 376293, 381279, 471658, 476751, 482423, 488877, 650373, 659213, 792059, 1508962, 1604957, 1624160

Appendix C
Threshold Sensitivity Analysis

Every benchmark is defined by one or two threshold rules applied to a continuous score. To verify that the chosen cutoffs are not arbitrary knife-edge values, each threshold was swept over a range bracketing the chosen value while all other parameters (sparsity floor, aggregation level, random seed) were held fixed. The sample size is $n = 25$ per level. Two criteria guided every threshold choice: (i) the qualifying pool must exceed the sample size at every targeted aggregation level, and (ii) the pool must vary smoothly across adjacent sweep steps so that small perturbations produce proportionate changes. Criterion (i) is violated by the No Temporal Structure benchmark at the institutional level at both resolutions (13 qualifying series at hourly, 19 at 10-minute), a structural constraint caused by the averaging effect that makes institutional-level series inherently autocorrelated. The full sweep notebooks are available in the attached source repository in the `sensitivity_analysis/` directory.

The following sections present per-benchmark sweep results and score distributions at both the hourly and 10-minute resolutions.

C.1 Seasonality

The sole parameter is the seasonal strength cutoff $F_S \geq \tau$ (Equation 1.3). The threshold was swept from 0.50 to 0.90 in steps of 0.05, with the 40 % sparsity floor applied first. After sparsity filtering, the eligible populations are 281 institutions, 535 subnets, and 42,706 IP addresses at the hourly resolution and 266, 468, and 34,729 at the 10-minute resolution.

Figure C.1 shows the F_S distribution over the post-sparsity population at both resolutions.

Figure C.2 shows the pool-size sweep. The pool decays monotonically with no step change near the chosen 0.70. At the hourly resolution, the institutional pool stays above 100 across $[0.50, 0.70]$ and drops to 90 at $\tau = 0.75$ and 36 at $\tau = 0.80$. The 10-minute resolution is tighter: the institutional pool is 63 at

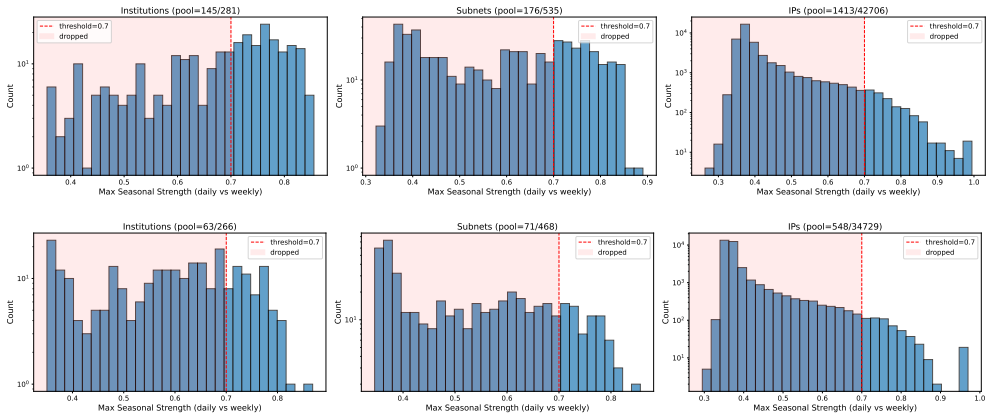


Figure C.1 Distribution of F_S over the post-sparsity population at the hourly (top) and 10-minute (bottom) resolutions. Bars to the right of the vertical line form the qualifying pool at the chosen $\tau = 0.70$.

$\tau = 0.70$ and 30 at $\tau = 0.75$, just above the sample size. The chosen 0.70 is therefore the highest value that keeps qualifying pools above $n = 25$ at every aggregation level and both resolutions.

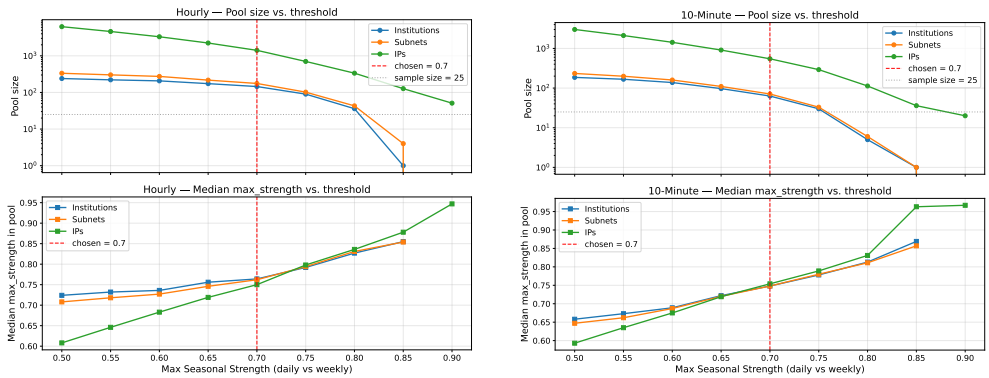


Figure C.2 Pool size (top panel, log scale) and median F_S in pool (bottom panel) as a function of τ at the hourly (left) and 10-minute (right) resolutions. Dashed vertical line: chosen $\tau = 0.70$. Dotted horizontal line: $n = 25$.

C.2 Data Drift

The qualifying rule combines two parameters: a relative-deviation cutoff τ applied to rel_dev(t) (Equation 4.1) and a persistence window p (days). For each series, define

$$S_p = \max_t \min_{s \in [t-p+1,\, t]} \text{rel_dev}(s),$$

the largest deviation amplitude that the series sustains over any contiguous p-day window. The qualifying rule reduces to $S_p \geq \tau$. Each parameter was swept separately while holding the other at its chosen value ($\tau = 0.30$, $p = 14$ days). The post-sparsity ($\geq 90\,\%$ active) population is 276 institutions, 507 subnets, and 21,451 IP addresses at the hourly resolution and 235, 373, and 7,353 at the 10-minute resolution.

Deviation sweep ($p = 14$ days fixed). Figure C.3 shows the distribution of the maximum 14-day-sustained deviation S_p across the post-sparsity population. Figure C.4 shows the pool-size sweep over $\tau \in [0.10, 0.60]$. At the chosen $\tau = 0.30$, the institution and subnet pools are 103 and 214, both at least 4× the sample size. Tightening to $\tau = 0.35$ reduces the institutional pool by 23 series and the subnet pool by 38, while relaxing to $\tau = 0.25$ gains 41 and 54 — smooth, proportionate changes.

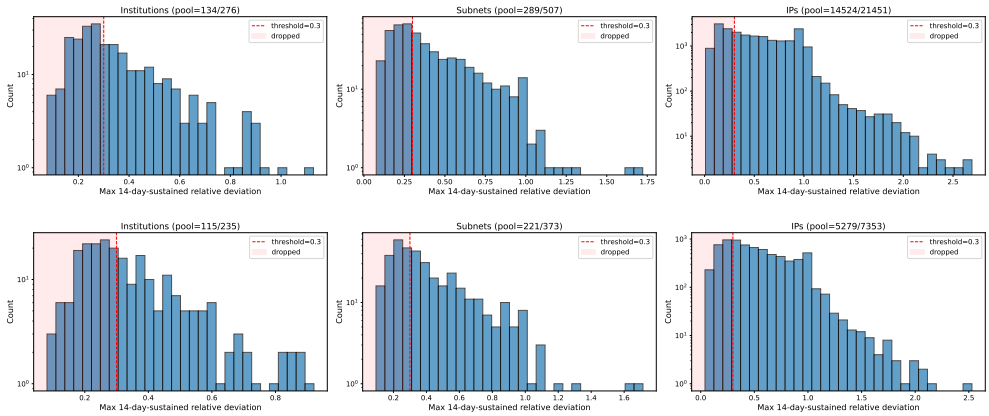


Figure C.3 Distribution of the maximum 14-day-sustained deviation S_p at the hourly (top) and 10-minute (bottom) resolutions.

Persistence sweep ($\tau = 0.30$ fixed). The persistence parameter p was swept over $\{7, 14, 21, 28\}$ days at fixed $\tau = 0.30$ (Figure C.5). The chosen $p = 14$ days corresponds to two complete weekly cycles, which is the minimum window that guarantees a deviation has survived the dominant weekly periodicity of CESNET traffic. Going from $7 \rightarrow 14$ days roughly halves the institutional pool (from 255 to 103) by filtering out transient bursts shorter than two weekly cycles. Going from $14 \rightarrow 21$ days collapses the institutional pool to 18, below the sample size of 25, making 14 days the longest persistence window that yields a pool above $n = 25$ at all aggregation levels. The IP-level pools are much larger (17,465 at $p = 7$, 10,541 at $p = 14$, 5,591 at $p = 21$) and decay more gradually because the post-sparsity IP population spreads its S_p mass across a wider range.

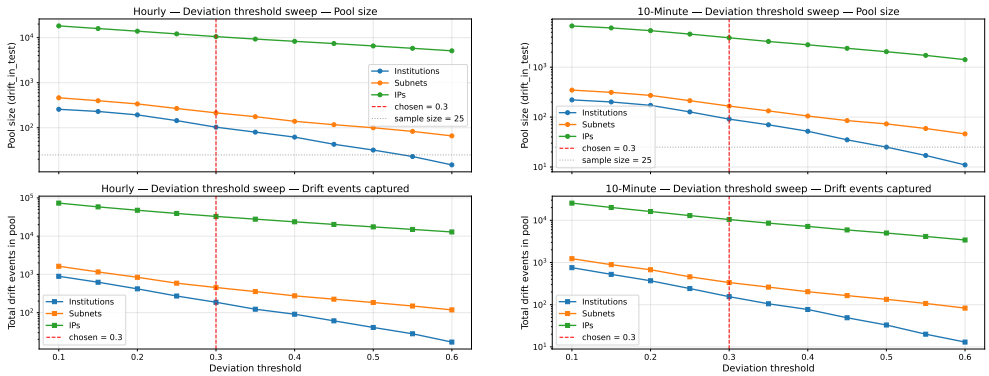


Figure C.4 Deviation-threshold sweep at the hourly (left) and 10-minute (right) resolutions, showing pool size and drift-event count as a function of τ. Dashed vertical line: chosen $\tau = 0.30$.

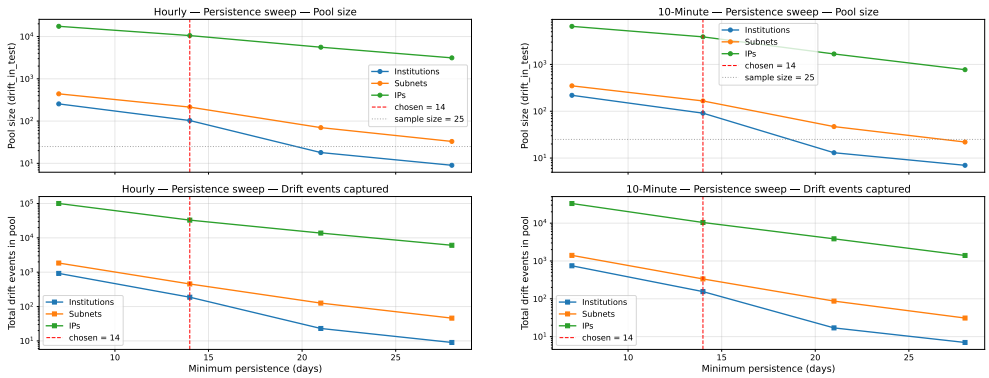


Figure C.5 Pool size as a function of persistence p at the hourly (left) and 10-minute (right) resolutions. Dashed vertical line: chosen $p = 14$ days.

Calendar-week diversity check. A potential failure mode of the joint qualifying rule is selecting 25 series that all contain the same network-wide drift event (described by the authors of the original dataset in [17]), which would make the benchmark a single-event test rather than a test of general robustness to drift. To rule this out, the calendar weeks containing at least one drift event were tabulated for the selected series at each aggregation level (Table C.1). At the hourly resolution, the 75 selected series (25×3 levels) produce 166 drift events spread across 33 of 40 calendar weeks. At the 10-minute resolution, the corresponding figures are 165 events over 32 weeks. No single week dominates the event count, which confirms that the joint qualifying rule captures a heterogeneous set of structural shifts rather than a single network-wide event.

Table C.1 Calendar-week coverage of the selected Data Drift series. The dataset window spans 40 calendar weeks. Events = distinct drift events in the 25 selected series at each level. Weeks = distinct calendar weeks containing at least one event.

Resolution	Inst. ev.	Inst. wk.	Sub. ev.	Sub. wk.	IP ev.	IP wk.
Hourly	39	12	45	17	82	30
10-minute	45	18	50	18	70	29

C.3 Periodic Spaces

The qualifying rule combines the ACF cutoff $\max \mathrm{ACF}(B_t)[\tau_{\mathrm{peak}}] \geq c$ with the activity band $[0.05, 0.95]$. The cutoff c was swept from 0.40 to 0.80 in steps of 0.025, and the chosen band was separately compared against the tighter alternative $[0.10, 0.90]$.

ACF cutoff sweep. Figure C.6 shows the distribution of the maximum prominent ACF value over the in-band population. For a binary series of length 6,718, the white-noise 95 % confidence bound on the ACF is $\pm 1.96/\sqrt{6{,}718} \approx \pm 0.024$. In variance terms, $\mathrm{ACF}^2 = 0.36$ means 36 % of the binary-series variance at the dominant lag is explained by periodic repetition.

Figure C.7 shows that the pool decays roughly exponentially as c increases. At the chosen $c = 0.60$, the pool is 5,684 hourly IP addresses and 1,580 at the 10-minute resolution — roughly 200× and 60× the sample size. Shifting by one step to $c = 0.575$ or $c = 0.625$ changes the hourly pool by +1,604 or −1,273, a proportionate change on a large base far from any knife-edge.

Activity-band sensitivity. Table C.2 compares the chosen activity band $[0.05, 0.95]$ against the tighter alternative $[0.10, 0.90]$ at fixed $c = 0.60$. Tightening the band reduces the in-band population by 26 % (hourly) and 39 % (10-minute), but the qualifying pool shrinks by only 11 % and 14 % respectively. An IP address whose binary indicator has a dominant weekly period-

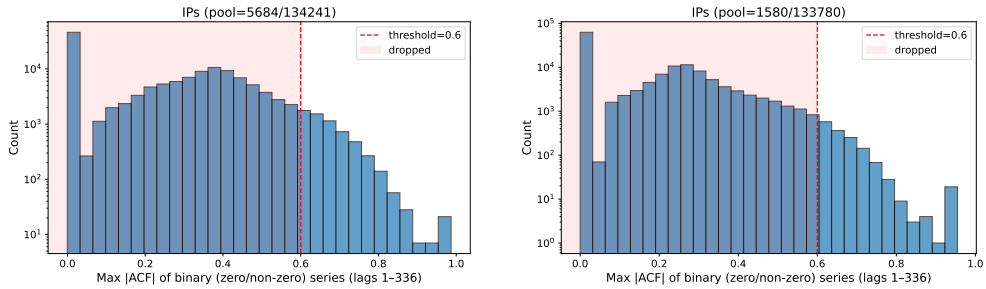


Figure C.6 Distribution of the maximum prominent ACF value over the in-band population at the hourly (left) and 10-minute (right) resolutions.

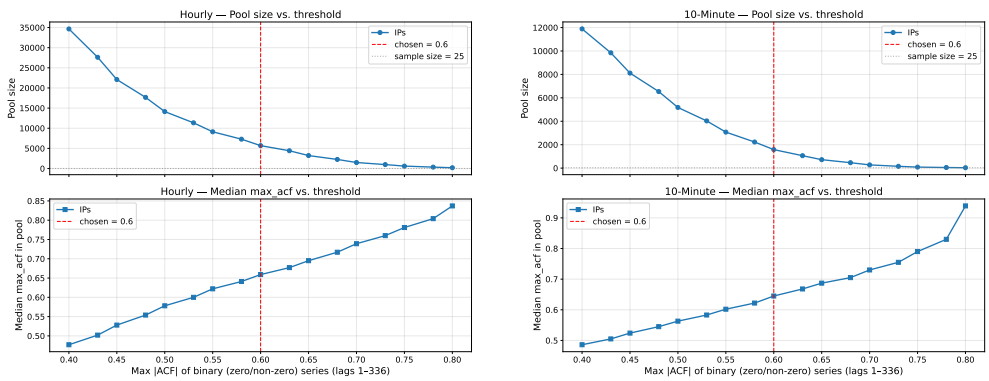


Figure C.7 Pool size (top panel, log scale) and median ACF (bottom panel) as a function of cutoff c at the hourly (left) and 10-minute (right) resolutions. Dashed vertical line: chosen $c = 0.60$.

icity (ACF $\geq$ 0.6) is, by construction, transmitting and silent in a regular pattern, so it cannot plausibly sit at the extreme ends of the activity ratio. The $[0.05, 0.95]$ band therefore acts purely as a guard against degenerate near-constant binary series, and the ACF cutoff is the operative selection criterion.

Table C.2 Periodic Spaces activity-band comparison at fixed $c = 0.60$. In-band counts the post-filtering population. Pool counts series passing both the band and the ACF cutoff.

Band	Hr. in-band	Hr. pool	10m in-band	10m pool
[0.05, 0.95] (chosen)	134 241	5 684	133 780	1 580
[0.10, 0.90] (tighter)	98 866	5 050	81 487	1 351
Change	−26 %	−11 %	−39 %	−14 %

C.4 Workstation Activity

The qualifying rule combines WHR $\geq w$ (Equation 4.2) with an activity-ratio floor of 0.23, computed by approximation of the minimal working hours in the dataset period. The WHR was swept from 0.25 to 0.70 in steps of 0.05, holding the activity ratio fixed. The post-activity-ratio population is 60,022 (hourly) and 49,513 (10-minute) IP addresses.

WHR sweep. Figure C.8 shows the WHR distribution and Figure C.9 shows the pool-size sweep. The pool decays steeply and monotonically as w increases. There is a notable step between $w = 0.30$ and $w = 0.35$, where requiring WHR ≥ 0.35 cuts the hourly pool from 54,958 to 11,387. This reflects that the bulk of sufficiently active IP addresses have moderate WHR (0.25–0.35), i.e., a workday-weighted but not exclusively working-hours-concentrated pattern. Above 0.35 the decay is smoother. At the chosen $w = 0.60$, the pool is 120 hourly and 34 ten-minute IP addresses. The 10-minute pool of 34 is the binding constraint: raising to $w = 0.65$ would leave only 7, well below the sample size.

Joint-binding analysis. The activity-ratio floor might appear non-binding for genuine workstations — a device active throughout office hours for most of the year should easily clear the bar. Table C.3 shows the contrary. At the hourly resolution, 24,113 IP addresses satisfy WHR ≥ 0.60 alone, but only 120 (0.5 %) also clear the activity-ratio floor. The 10-minute resolution is even tighter: 20,479 pass the WHR cutoff but only 34 (0.2 %) clear both. Any IP address that is active for only 50–100 hours over the eight-month observation window can trivially achieve WHR > 0.9 if those isolated hours happen to fall during business hours. Such a series provides almost no forecasting signal. The activity-ratio floor enforces a minimum data volume, and the WHR then

selects for sustained working-hours concentration among the remaining well-populated series. Both thresholds address distinct failure modes.

Table C.3 Joint effect of the two filters at $w = 0.60$. "WHR only" counts IP addresses satisfying WHR ≥ 0.60 regardless of activity ratio. "Both" adds the activity-ratio floor ≥ 0.23.

Resolution	WHR only	Both	Removed	Removed (%)
Hourly	24 113	120	23 993	99.5 %
10-minute	20 479	34	20 445	99.8 %

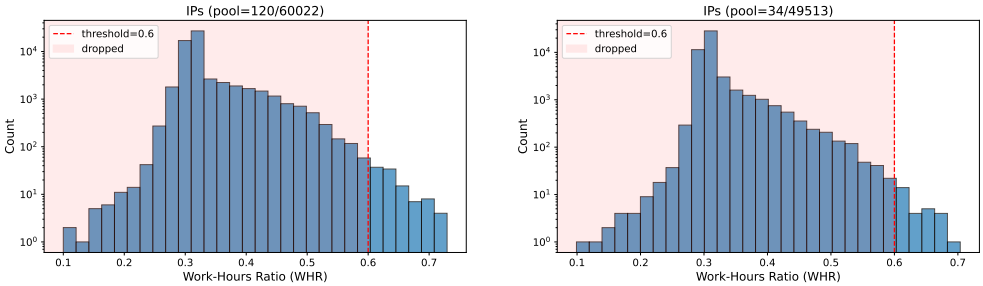


Figure C.8 Distribution of WHR over the post-activity-ratio population at the hourly (left) and 10-minute (right) resolutions.

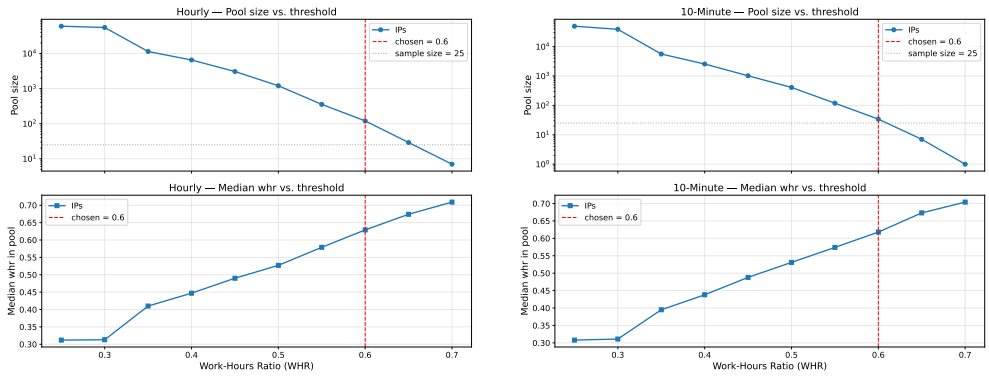


Figure C.9 Pool size (top panel, log scale) and median WHR (bottom panel) as a function of w at the hourly (left) and 10-minute (right) resolutions. Dashed vertical line: chosen $w = 0.60$.

C.5 No Temporal Structure

The sole parameter is $\max|\text{ACF}| \leq c$ over lags 1–168 (hourly) or 1–1,008 (10-minute). The cutoff was swept from 0.05 to 0.30, with the 40 % sparsity floor applied first. After sparsity filtering, the eligible populations are 281 institutions, 535 subnets, and 42,706 IP addresses at the hourly resolution and 266, 468, and 34,729 at the 10-minute resolution.

Figure C.10 shows the $\max|\text{ACF}|$ distribution and Figure C.11 shows the pool-size sweep. The institution and subnet pools grow smoothly as c increases, with no abrupt jump near the chosen $c = 0.10$. At the hourly resolution, 5 institutions qualify at $c = 0.05$, growing to 13 at $c = 0.10$ and 50 at $c = 0.30$. The corresponding subnet counts are 25 at $c = 0.10$ and 162 at $c = 0.30$. The IP-level pools remain on the order of 10^4 across the entire range.

The institutional pool is small by structural necessity. Aggregating over an entire institution mixes traffic from hundreds or thousands of individual endpoints. Even when individual endpoints are irregular, their combined activity produces regular daily and weekly cycles through an averaging effect. Even relaxing the cutoff to 0.30 yields only 50 hourly institutions out of 281 post-sparsity candidates. The chosen $c = 0.10$ corresponds to $r^2 \leq 0.01$ at any individual lag — less than 1 % of variance explained, falling below Cohen's conventional threshold for a "small" correlation effect [66]. With series lengths of $N = 6{,}718$, the white-noise 95 % confidence bound on the sample ACF is $\pm 1.96/\sqrt{6{,}718} \approx \pm 0.024$. The threshold of 0.10 is approximately 4× this noise floor. It is strict enough to exclude any series with pronounced periodic structure while remaining above the noise floor to allow for estimation noise in the sample ACF. Relaxing to $c = 0.15$ or $c = 0.20$ would admit series with autocorrelation values well above the 95 % confidence bound at individual lags, weakening the benchmark's defining property.

At the institutional level, the benchmark takes all 13 (hourly) or 19 (10-minute) qualifying series rather than sub-sampling 25. This is the only benchmark where criterion (i) is violated. The sub-sampling rule is applied normally at the subnet and IP levels, where the qualifying pool comfortably exceeds 25. The small institutional pool is itself informative: it shows that finding institution-level series without autocorrelation is genuinely difficult in this dataset, which underlines why the benchmark functions as a meaningful diagnostic rather than a trivially satisfied condition.

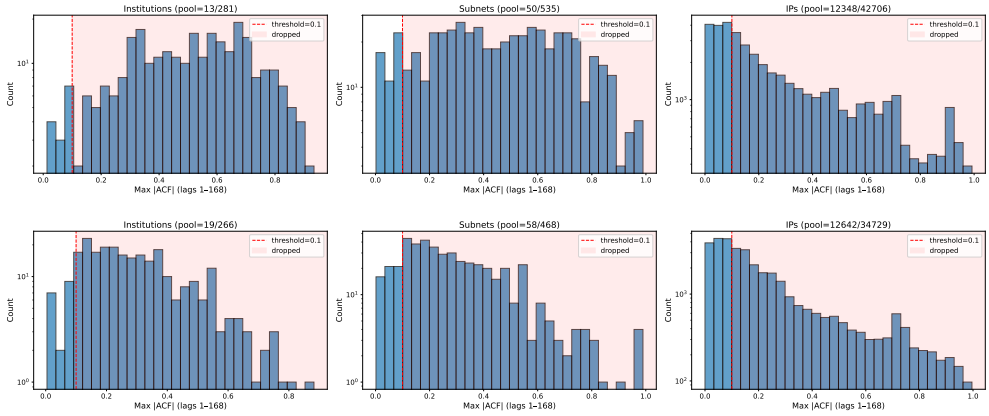


Figure C.10 Distribution of max |ACF| over the post-sparsity population at the hourly (top) and 10-minute (bottom) resolutions. Institutions have modal max |ACF| in $[0.3, 0.7]$ at hourly resolution, confirming that institutional-level series are predominantly autocorrelated.

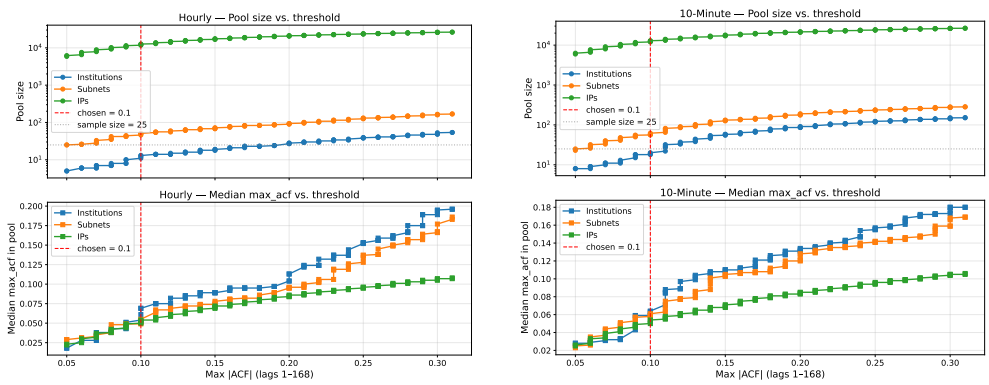


Figure C.11 Pool size (top panel, log scale) and median max |ACF| (bottom panel) as a function of c at the hourly (left) and 10-minute (right) resolutions. Dashed vertical line: chosen $c = 0.10$. Dotted horizontal line: $n = 25$.

Appendix D

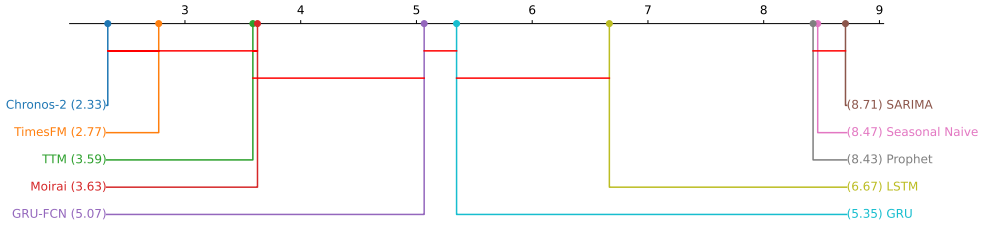
Friedman Test Statistics and Critical Difference Diagrams

Table D.1 reports the Friedman test statistics for all 20 benchmark–configuration combinations. All tests reject the null hypothesis of equal model ranks at $\alpha = 0.05$ after Holm–Bonferroni correction.

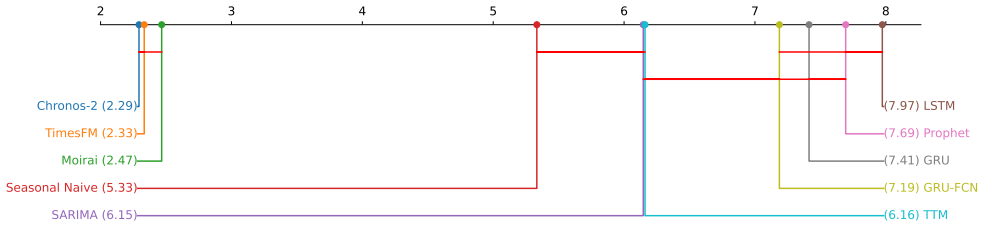
The following pages show the Nemenyi post-hoc critical difference diagrams for all 20 tests, grouped by benchmark. Two diagrams (Data Drift h/744/168 and Workstation Activity m/1008/144) are shown in Section 7.2 and are repeated here for completeness. Connected models are not significantly different at $\alpha = 0.05$.

Table D.1 Friedman χ^2 statistics per benchmark and configuration. All p-values are Holm–Bonferroni corrected within each configuration. $k = 10$ models in all tests.

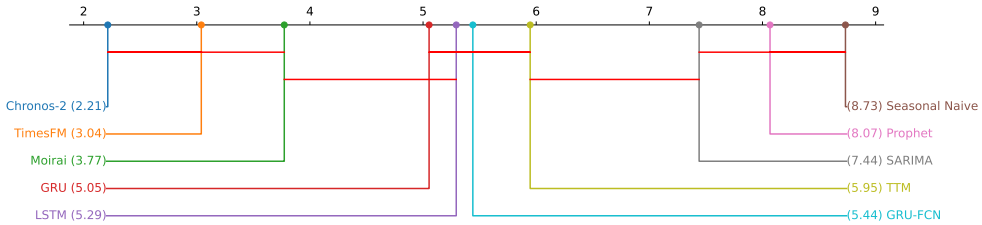
Configuration	**Benchmark**	n	χ^2	p **(Holm)**
h/168/24	Drift	75	440.63	1.2×10^{-88}
	Seasonality	75	487.23	1.6×10^{-98}
	Periodic Spaces	25	106.87	6.4×10^{-19}
	Workstation	25	164.20	2.0×10^{-30}
	No Temp. Str.	63	334.06	4.6×10^{-66}
h/744/168	Drift	75	391.31	3.9×10^{-78}
	Seasonality	75	445.79	1.1×10^{-89}
	Periodic Spaces	25	107.82	4.1×10^{-19}
	Workstation	25	187.79	2.4×10^{-35}
	No Temp. Str.	63	272.80	4.5×10^{-53}
m/288/144	Drift	75	336.16	2.2×10^{-66}
	Seasonality	75	261.99	8.8×10^{-51}
	Periodic Spaces	25	109.75	1.7×10^{-19}
	Workstation	25	123.69	4.7×10^{-22}
	No Temp. Str.	69	375.23	1.3×10^{-74}
m/1008/144	Drift	75	395.48	5.1×10^{-79}
	Seasonality	75	414.33	6.0×10^{-83}
	Periodic Spaces	25	90.36	1.4×10^{-15}
	Workstation	25	199.34	9.1×10^{-38}
	No Temp. Str.	69	387.72	1.7×10^{-77}



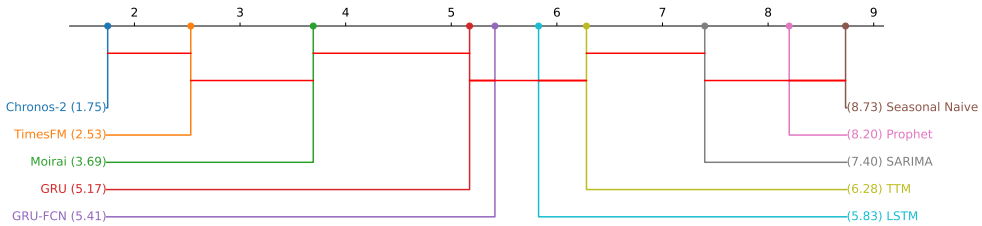
(a) h/168/24 ($n = 75$)



(b) h/744/168 ($n = 75$)

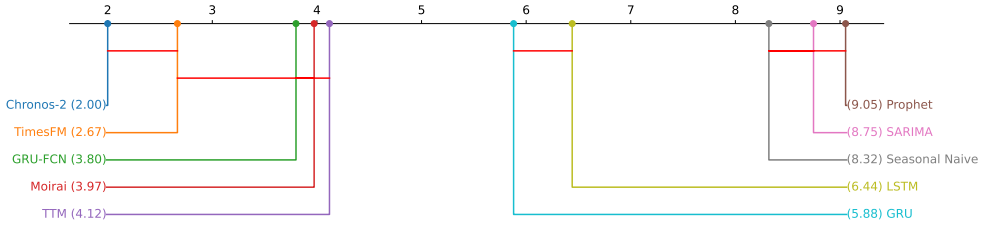


(c) m/288/144 ($n = 75$)

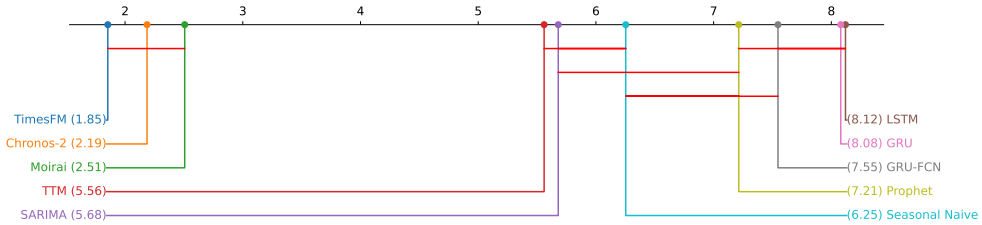


(d) m/1008/144 ($n = 75$)

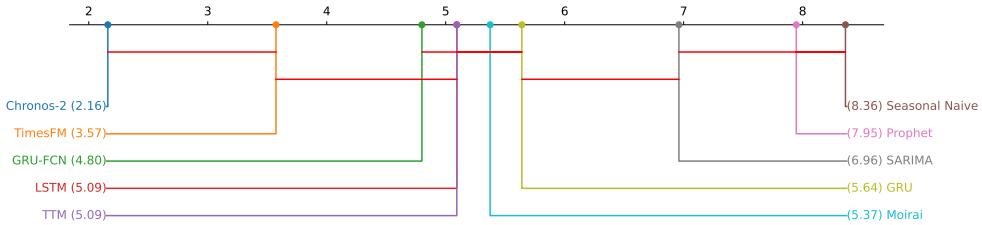
Figure D.1 CD diagrams: Data Drift across all four configurations.



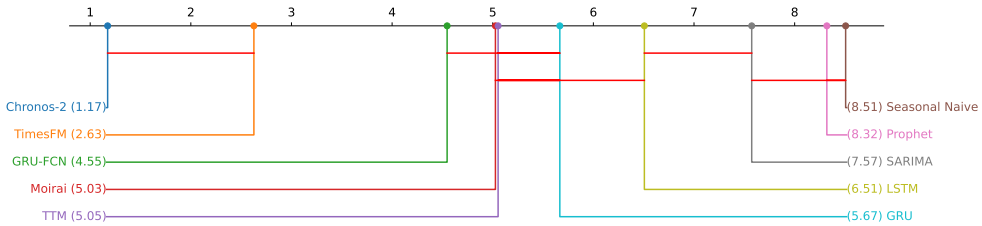
(a) h/168/24 ($n = 75$)



(b) h/744/168 ($n = 75$)

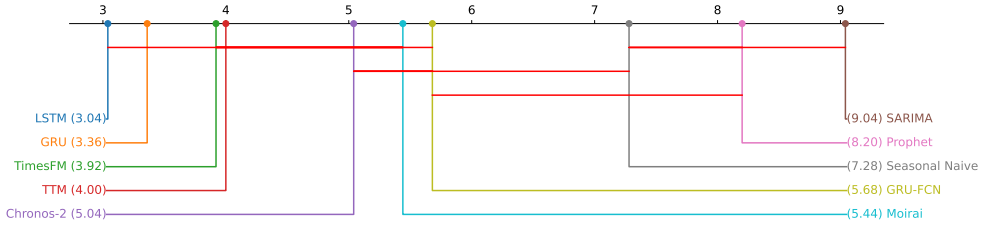


(c) m/288/144 ($n = 75$)

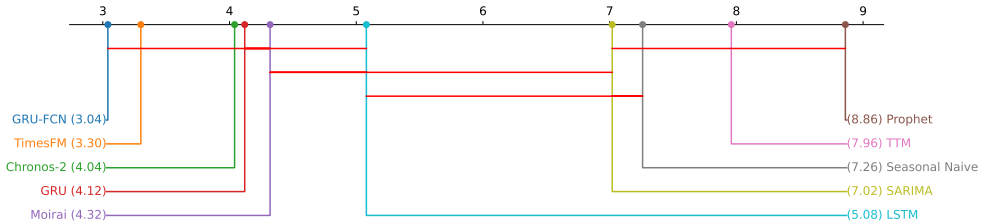


(d) m/1008/144 ($n = 75$)

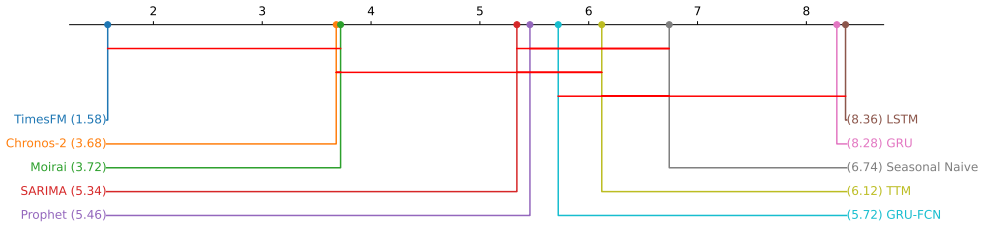
Figure D.2 CD diagrams: Seasonality across all four configurations.



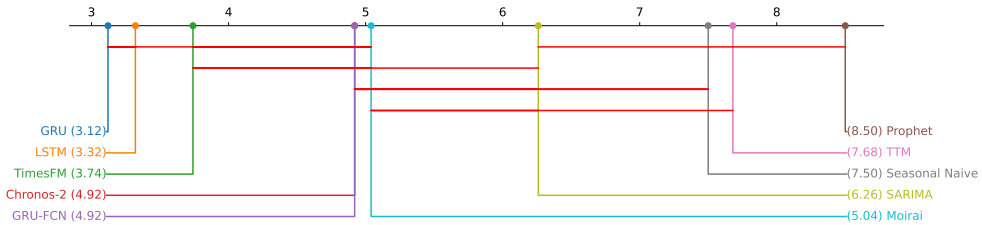
(a) h/168/24 ($n = 25$)



(b) h/744/168 ($n = 25$)

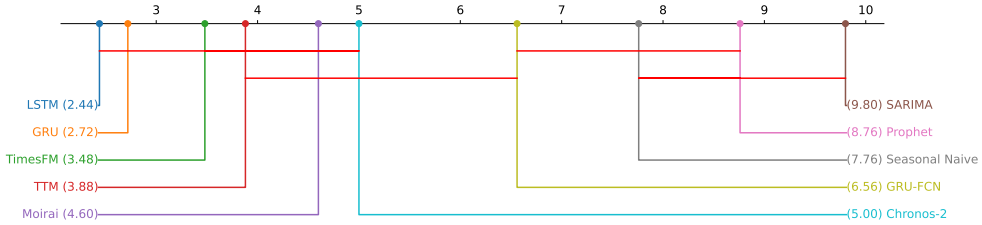


(c) m/288/144 ($n = 25$)

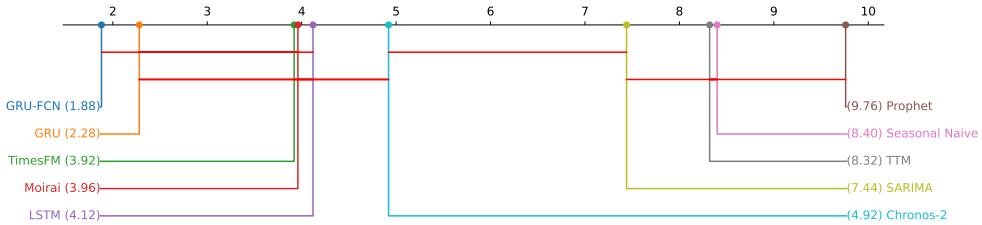


(d) m/1008/144 ($n = 25$)

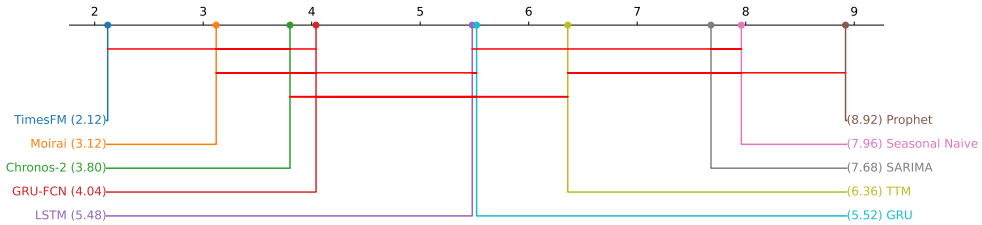
Figure D.3 CD diagrams: Periodic Spaces across all four configurations.



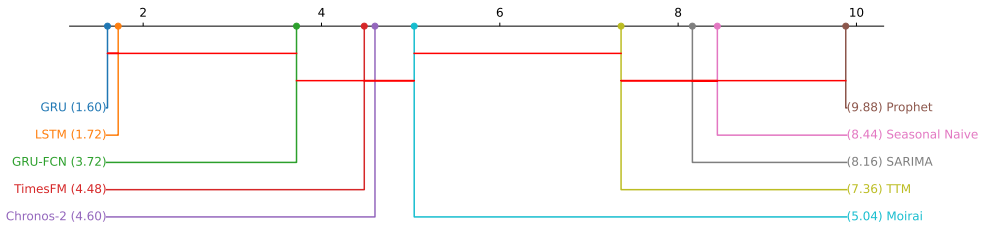
(a) h/168/24 ($n = 25$)



(b) h/744/168 ($n = 25$)

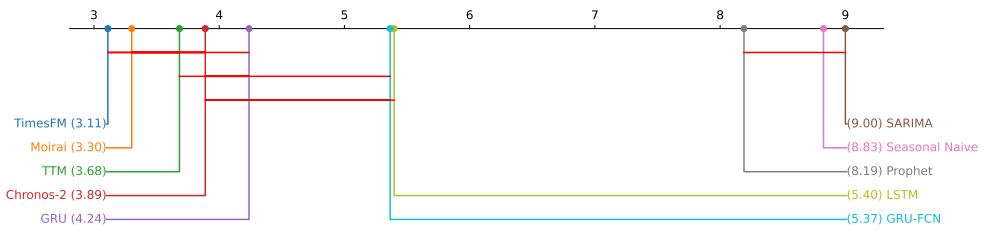


(c) m/288/144 ($n = 25$)

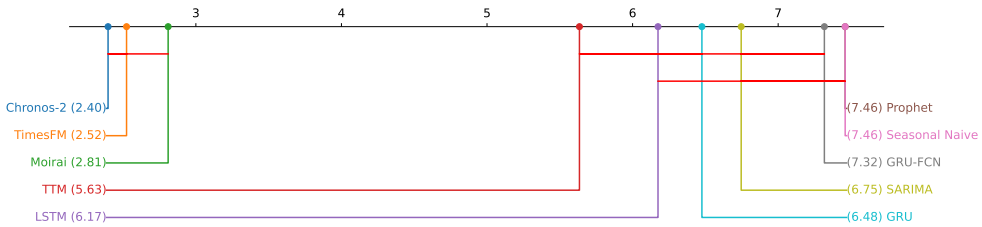


(d) m/1008/144 ($n = 25$)

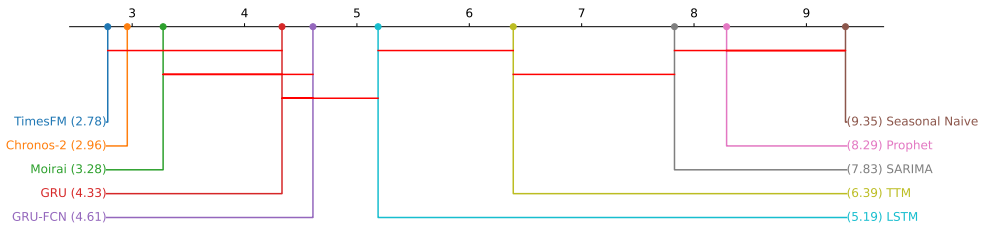
Figure D.4 CD diagrams: Workstation Activity across all four configurations.



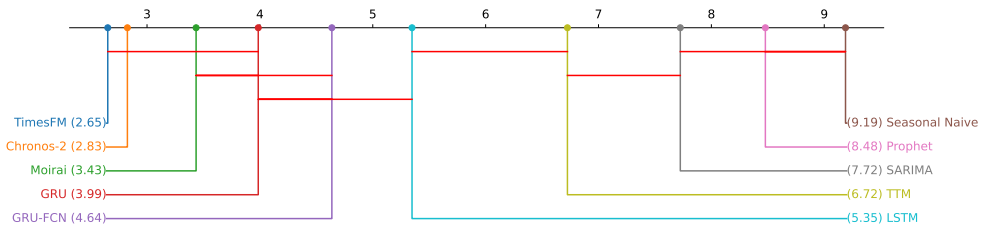
(a) h/168/24 ($n = 63$)



(b) h/744/168 ($n = 63$)



(c) m/288/144 ($n = 69$)



(d) m/1008/144 ($n = 69$)

Figure D.5 CD diagrams: No Temporal Structure across all four configurations.

Appendix E

Inference Speed

Table E.1 reports the total wall-clock time per single series (tune and predict over the full test split) for each model and configuration. For SARIMA and Prophet, the tune step is a per-window refit. For the deep learning models, it is a one-time per-series training pass. Foundation models perform only forward passes and have no tune step. All evaluations were executed on MetaCentrum, the Czech national HPC infrastructure. Each SARIMA, Prophet, and Seasonal Naïve job was allocated a CPU-only node with 4 cores and 16 GB of RAM. Each foundation-model and deep-learning job was allocated a GPU node with 4 cores, 32 GB of RAM, and one GPU with 16 GB of memory. The specific GPU model assigned by the PBS scheduler varies across submissions, so the reported times reflect typical—not deterministic—wall-clock costs on this class of hardware.

Table E.1 Total wall-clock time per series (seconds) including tune and predict over the full test split. Models are ordered by average time.

Model	h/168/24	h/744/168	m/288/144	m/1008/144
S. Naïve	≈0	≈0	≈0	≈0
Moirai	0.16	0.12	0.22	0.25
TimesFM	0.31	0.26	0.21	0.20
Chronos-2	0.28	0.06	0.18	0.22
TTM	0.46	0.08	0.21	0.19
GRU	0.69	0.21	0.82	1.07
LSTM	0.75	0.33	0.94	2.02
GRU-FCN	2.88	0.22	1.60	1.68
Prophet	4.74	1.73	5.54	13.04
SARIMA	29.50	16.63	14.45	40.65

Bibliography

1. CISCO SYSTEMS. *Cisco Annual Internet Report (2018–2023) White Paper* [online]. 2020. [visited on 2026-03-25]. Tech. rep. Cisco Systems. Available from: https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html.
2. FERREIRA, Gabriel O. et al. Forecasting network traffic: A survey and tutorial with open-source comparative evaluation. *IEEE Access*. 2023, vol. 11, pp. 6018–6044. Available from DOI: 10.1109/ACCESS.2023.3236261.
3. BOX, George E. P.; JENKINS, Gwilym M.; REINSEL, Gregory C.; LJUNG, Greta M. *Time Series Analysis: Forecasting and Control*. 5th ed. Hoboken, NJ: John Wiley & Sons, 2015. ISBN 978-1-118-67502-1.
4. LIM, Bryan; ZOHREN, Stefan. Time-series forecasting with deep learning: a survey. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*. 2021, vol. 379, no. 2194, p. 20200209. ISSN 1364-503X. Available from DOI: 10.1098/rsta.2020.0209.
5. BENIDIS, Konstantinos et al. Deep Learning for Time Series Forecasting: Tutorial and Literature Survey. *ACM Comput. Surv.* 2022, vol. 55, no. 6, 121:1–121:36. ISSN 0360-0300. Available from DOI: 10.1145/3533382.
6. LIANG, Yuxuan et al. Foundation Models for Time Series Analysis: A Tutorial and Survey. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: Association for Computing Machinery, 2024, pp. 6555–6565. KDD '24. Available from DOI: 10.1145/3637528.3671451.
7. WEN, Qingsong et al. Transformers in time series: a survey. In: ELKIND, Edith (ed.). *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. Macao, S.A.R.: International Joint Con-

ferences on Artificial Intelligence Organization, 2023. IJCAI '23. ISBN 978-1-956792-03-4. Available from DOI: 10.24963/ijcai.2023/759.

8. DAS, Abhimanyu; KONG, Weihao; SEN, Rajat; ZHOU, Yichen. A decoder-only foundation model for time-series forecasting. In: *Proceedings of the 41st International Conference on Machine Learning*. Vienna, Austria: JMLR.org, 2024. ICML'24.
9. ANSARI, Abdul Fatir et al. *Chronos-2: From Univariate to Universal Forecasting* [online]. 2025. [visited on 2026-03-25]. Available from arXiv: 2510.15821 [cs.LG].
10. LIU, Chenghao et al. *Moirai 2.0: When Less Is More for Time Series Forecasting* [online]. 2025. [visited on 2026-03-25]. Available from arXiv: 2511.11698 [cs.LG].
11. LIU, Fan; FARKIANI, Behrooz; CROWLEY, Patrick. *Time-Series Foundation Models for ISP Traffic Forecasting*. 2025. Available from arXiv: 2511.17529 [cs.NI].
12. MAKRIDAKIS, Spyros; SPILIOTIS, Evangelos; ASSIMAKOPOULOS, Vassilios. The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting* [online]. 2020, vol. 36, no. 1, pp. 54–74 [visited on 2026-03-26]. ISSN 0169-2070. Available from DOI: 10.1016/j.ijforecast.2019.04.014.
13. GODAHEWA, Rakshitha et al. Monash Time Series Forecasting Archive. In: VANSCHOREN, J.; YEUNG, S. (eds.). *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks* [online]. Curran Associates, 2021, vol. 1 [visited on 2026-03-26]. Available from: https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/eddea82ad2755b24c4e168c5fc2ebd40-Paper-round2.pdf.
14. AKSU, Taha et al. *GIFT-Eval: A Benchmark For General Time Series Forecasting Model Evaluation* [online]. 2024. [visited on 2026-03-26]. Available from arXiv: 2410.10393 [cs.LG].
15. KANG, Yanfei; HYNDMAN, Rob J.; SMITH-MILES, Kate. Visualising forecasting algorithm performance using time series instance spaces. *International Journal of Forecasting*. 2017, vol. 33, no. 2, pp. 345–358. Available from DOI: 10.1016/j.ijforecast.2016.09.004.
16. MAKRIDAKIS, Spyros; SPILIOTIS, Evangelos; ASSIMAKOPOULOS, Vassilios. M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting* [online]. 2022, vol. 38, no. 4, pp. 1346–1364 [visited on 2026-03-25]. ISSN 0169-2070. Available from DOI: 10.1016/j.ijforecast.2021.11.013.

17. KOUMAR, Josef; HYNEK, Karel; ČEJKA, Tomáš; ŠIŠKA, Pavel. CESNET-TimeSeries24: Time Series Dataset for Network Traffic Anomaly Detection and Forecasting. *Scientific Data* [online]. 2025, vol. 12, no. 1, p. 338 [visited on 2026-03-26]. ISSN 2052-4463. Available from DOI: 10.1038/s41597-025-04603-x.

18. HOFSTEDE, Rick et al. Flow Monitoring Explained: From Packet Capture to Data Analysis With NetFlow and IPFIX. *IEEE Communications Surveys & Tutorials*. 2014, vol. 16, no. 4, pp. 2037–2064. Available from DOI: 10.1109/COMST.2014.2321898.

19. CLAISE, Benoit; TRAMMELL, Brian; AITKEN, Paul. *Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information* [RFC 7011]. 2013. [visited on 2026-05-09]. Available from: https://www.rfc-editor.org/rfc/rfc7011.

20. CLAISE, Benoit. *Cisco Systems NetFlow Services Export Version 9* [RFC 3954]. 2004. [visited on 2026-05-09]. Available from: https://www.rfc-editor.org/rfc/rfc3954.

21. CESNET. *ipfixprobe: IPFIX Flow Exporter* [online]. 2024. [visited on 2026-05-09]. Available from: https://github.com/CESNET/ipfixprobe.

22. KOUMAR, Josef; SMOLEŇ, Timotej; JEŘÁBEK, Kamil; ČEJKA, Tomáš. Comparative Analysis of Deep Learning Models for Real-World ISP Network Traffic Forecasting. *IEEE Transactions on Network and Service Management*. 2026, vol. 23, pp. 715–728. Available from DOI: 10.1109/TNSM.2025.3636557.

23. HYNDMAN, Rob J.; ATHANASOPOULOS, George. *Forecasting: Principles and Practice* [online]. 3rd ed. Melbourne, Australia: OTexts, 2021 [visited on 2026-03-26]. Available from: https://otexts.com/fpp3/. Online open-access textbook.

24. BARTLETT, M. S. On the Theoretical Specification and Sampling Properties of Autocorrelated Time-Series. *Journal of the Royal Statistical Society*. 1946, vol. 8, no. 1, pp. 27–41. Available from DOI: 10.2307/2983611.

25. CLEVELAND, Robert B.; CLEVELAND, William S.; MCRAE, Jean E.; TERPENNING, Irma. STL: A Seasonal-Trend Decomposition Procedure Based on Loess. *Journal of Official Statistics*. 1990, vol. 6, no. 1, pp. 3–33.

26. CORTEZ, Paulo; RIO, Miguel; ROCHA, Miguel; SOUSA, Pedro. Multi-scale Internet traffic forecasting using neural networks and time series methods. *Expert Systems* [online]. 2012, vol. 29, no. 2, pp. 143–155 [visited on 2026-03-26]. Available from DOI: 10.1111/j.1468-0394.2010.00568.x.

27. WANG, Xiaozhe; SMITH, Kate; HYNDMAN, Rob. Characteristic-Based Clustering for Time Series Data. *Data Mining and Knowledge Discovery*. 2006, vol. 13, no. 3, pp. 335–364. Available from DOI: 10.1007/s10618-005-0039-x.

28. GAMA, João et al. A Survey on Concept Drift Adaptation. *ACM Computing Surveys*. 2014, vol. 46, no. 4, 44:1–44:37. Available from DOI: 10.1145/2523813.

29. AOUEDI, Ons; LE, Van An; PIAMRAT, Kandaraj; JI, Yusheng. Deep Learning on Network Traffic Prediction: Recent Advances, Analysis, and Future Directions. *ACM Computing Surveys* [online]. 2025, vol. 57, no. 6 [visited on 2026-03-27]. Available from DOI: 10.1145/3703447.

30. TAYLOR, Sean J.; LETHAM, Benjamin. Forecasting at Scale. *The American Statistician*. 2018, vol. 72, no. 1, pp. 37–45. Available from DOI: 10.1080/00031305.2017.1380080.

31. HOCHREITER, Sepp; SCHMIDHUBER, Jürgen. Long Short-Term Memory. *Neural Computation*. 1997, vol. 9, no. 8, pp. 1735–1780. Available from DOI: 10.1162/neco.1997.9.8.1735.

32. CHO, Kyunghyun et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. Doha, Qatar: Association for Computational Linguistics, 2014, pp. 1724–1734. Available from DOI: 10.3115/v1/D14-1179.

33. KARIM, Fazle; MAJUMDAR, Somshubra; DARABI, Houshang; HARFORD, Samuel. Multivariate LSTM-FCNs for Time Series Classification. *Neural Networks*. 2019, vol. 116, pp. 237–245. Available from DOI: 10.1016/j.neunet.2019.04.014.

34. HYNDMAN, Rob J.; KOEHLER, Anne B. Another look at measures of forecast accuracy. *International Journal of Forecasting*. 2006, vol. 22, no. 4, pp. 679–688. ISSN 0169-2070. Available from DOI: 10.1016/j.ijforecast.2006.03.001.

35. MAKRIDAKIS, Spyros; HIBON, Michèle. The M3-Competition: results, conclusions and implications. *International Journal of Forecasting*. 2000, vol. 16, no. 4, pp. 451–476. Available from DOI: 10.1016/S0169-2070(00)00057-1.

36. ZHANG, Yin; ROUGHAN, Matthew; DUFFIELD, Nick; GREENBERG, Albert. Fast Accurate Computation of Large-Scale IP Traffic Matrices from Link Loads. In: *Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. 2003, pp. 206–217. SIGMETRICS '03. Available from DOI: 10.1145/781027.781053.

37. UHLIG, Steve; QUOITIN, Bruno; LEPROPRE, Jean; BALON, Simon. Providing Public Intradomain Traffic Matrices to the Research Community. *SIGCOMM Computer Communication Review*. 2006, vol. 36, no. 1, pp. 83–86. Available from DOI: 10.1145/1111322.1111341.

38. FONTUGNE, Romain; BORGNAT, Pierre; ABRY, Patrice; FUKUDA, Kensuke. MAWILab: Combining Diverse Anomaly Detectors for Automated Anomaly Labeling and Performance Benchmarking. In: *ACM CoNEXT '10*. Philadelphia, PA, 2010.

39. CAIDA. *The CAIDA Anonymized Internet Traces* [online]. [N.d.]. [visited on 2026-05-10]. Available from: https://www.caida.org/catalog/datasets/passive_dataset_download/.

40. WU, Haixu et al. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In: *International Conference on Learning Representations* [online]. 2023 [visited on 2026-05-10]. Available from: https://arxiv.org/abs/2210.02186.

41. QIU, Xiangfei et al. TFB: Towards Comprehensive and Fair Benchmarking of Time Series Forecasting Methods. *Proceedings of the VLDB Endowment*. 2024, vol. 17, no. 9, pp. 2363–2377. Available from DOI: 10.14778/3665844.3665863.

42. SHCHUR, Oleksandr et al. *fev-bench: A Realistic Benchmark for Time Series Forecasting* [online]. 2026. [visited on 2026-03-26]. Available from arXiv: 2509.26468 [cs.LG].

43. LI, Zhe et al. TSFM-Bench: A Comprehensive and Unified Benchmark of Foundation Models for Time Series Forecasting. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2025, pp. 5595–5606. Available from DOI: 10.1145/3711896.3737442.

44. LIU, Xu et al. LargeST: A Benchmark Dataset for Large-Scale Traffic Forecasting. In: *Advances in Neural Information Processing Systems* [online]. 2023, vol. 36 [visited on 2026-05-10]. Available from: https://arxiv.org/abs/2306.08259.

45. YIN, Du et al. *XXLTraffic: Expanding and Extremely Long Traffic Dataset for Ultra-Dynamic Forecasting Challenges* [online]. 2025. [visited on 2026-05-10]. Available from arXiv: 2406.12693 [cs.LG].

46. QIAO, Zhongzheng et al. *It's TIME: Towards the Next Generation of Time Series Forecasting Benchmarks* [online]. 2026. [visited on 2026-05-10]. Available from arXiv: 2602.12147 [cs.LG].

47. EKAMBARAM, Vijay et al. Tiny Time Mixers (TTMs): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series. In: *Advances in Neural Information Processing Systems*. 2024. Available from arXiv: 2401.03955 [cs.LG].

48. KUREŠ, Milan; KOUMAR, Josef; HYNEK, Karel. CESNET TS-Zoo: A Library for Reproducible Analysis of Network Traffic Time Series. In: *2025 21st International Conference on Network and Service Management (CNSM)*. 2025, pp. 1–5. Available from DOI: 10.23919/CNSM67658.2025.11297513.

49. VIRTANEN, Pauli et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*. 2020, vol. 17, pp. 261–272. Available from DOI: 10.1038/s41592-019-0686-2.

50. TASHMAN, Leonard J. Out-of-sample tests of forecasting accuracy: an analysis and review. *International Journal of Forecasting* [online]. 2000, vol. 16, no. 4, pp. 437–450 [visited on 2026-03-26]. ISSN 0169-2070. Available from DOI: 10.1016/S0169-2070(00)00065-0.

51. KIM, Taesung et al. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. In: *International Conference on Learning Representations* [online]. 2022 [visited on 2026-04-05]. Available from: https://api.semanticscholar.org/CorpusID:251647808.

52. ARMSTRONG, J. Scott. Evaluating Forecasting Methods. In: ARMSTRONG, J. Scott (ed.). *Principles of Forecasting: A Handbook for Researchers and Practitioners*. Boston, MA: Springer US, 2001, pp. 443–472. Available from DOI: 10.1007/978-0-306-47630-3_20.

53. DEMŠAR, Janez. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*. 2006, vol. 7, pp. 1–30.

54. HOLM, Sture. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*. 1979, vol. 6, no. 2, pp. 65–70.

55. MCKINNEY, Wes. Data Structures for Statistical Computing in Python. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. Available from DOI: 10.25080/Majora-92bf1922-00a.

56. HARRIS, Charles R. et al. Array programming with NumPy. *Nature*. 2020, vol. 585, pp. 357–362. Available from DOI: 10.1038/s41586-020-2649-2.

57. PASZKE, Adam et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *Advances in Neural Information Processing Systems*. 2019, vol. 32, pp. 8024–8035.

58. OGUIZA, Ignacio. *tsai – A State-of-the-Art Deep Learning Library for Time Series and Sequential Data* [online]. 2022. [visited on 2026-04-07]. Available from: https://github.com/timeseriesAI/tsai.

59. HUNTER, John D. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*. 2007, vol. 9, no. 3, pp. 90–95. Available from DOI: 10.1109/MCSE.2007.55.

60. WASKOM, Michael L. seaborn: statistical data visualization. *Journal of Open Source Software*. 2021, vol. 6, no. 60, p. 3021. Available from DOI: 10.21105/joss.03021.

61. PLOTLY TECHNOLOGIES. *Plotly: Collaborative Data Science* [online]. 2015. [visited on 2026-04-07]. Available from: https://plot.ly.

62. IBM GRANITE TEAM. *Granite-TimeSeries-TTM-R2: Model Card* [online]. 2025. [visited on 2026-05-12]. Available from: https://huggingface.co/ibm-granite/granite-timeseries-ttm-r2. Hugging Face model card.

63. CROSTON, J. D. Forecasting and Stock Control for Intermittent Demands. *Journal of the Operational Research Society*. 1972, vol. 23, no. 3, pp. 289–303. Available from DOI: 10.1057/jors.1972.50.

64. SYNTETOS, Aris A.; BOYLAN, John E. The Accuracy of Intermittent Demand Estimates. *International Journal of Forecasting*. 2005, vol. 21, no. 2, pp. 303–314. Available from DOI: 10.1016/j.ijforecast.2004.10.001.

65. TEUNTER, Ruud H.; SYNTETOS, Aris A.; BABAI, M. Zied. Intermittent Demand: Linking Forecasting to Inventory Obsolescence. *European Journal of Operational Research*. 2011, vol. 214, no. 3, pp. 606–615. Available from DOI: 10.1016/j.ejor.2011.05.018.

66. COHEN, Jacob. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988. ISBN 978-0-8058-0283-2.

Attachment contents

```
/
├── README.md ..... overview
├── LICENSE .....
├── text/
│   ├── skacel-thesis.pdf ..... thesis text
│   ├── declaration.pdf ..... declaration of authorship
│   └── src/ ..... thesis LaTeX source
├── NetTS-experiments/ ..... benchmark construction
│   ├── README.md
│   ├── config.py ..... benchmark parameters and aggregation configs
│   ├── utils.py ..... data loading and helpers
│   ├── selection.py ..... threshold + random sampling from scores
│   ├── scoring/ ..... per-property score functions
│   ├── scores/ ..... pre-computed score CSVs
│   ├── selected_ids/ ..... curated series identifiers
│   ├── benchmarks/ ..... benchmark verification notebooks
│   ├── exploration/ ..... data exploration notebooks
│   ├── sensitivity_analysis/ ..... threshold sensitivity sweeps
│   ├── metacentrum/ ..... PBS submission for HPC scoring
│   └── requirements.txt
└── NetTS-eval/ ..... evaluation framework
    ├── README.md
    ├── nettseval/ ..... Python package (models, evaluation, metrics)
    ├── run_evaluation.py ..... evaluation CLI entry point
    ├── save_benchmarks.py .... benchmark data export from cesnet-tszoo
    ├── benchmarks/ ..... exported benchmark data (CSV/Parquet)
    ├── results/ ..... per-model evaluation outputs
    ├── result_exploration/ .. analysis pipeline (tables, plots, notebooks)
    ├── metacentrum/ ..... PBS submission for HPC evaluation
    └── requirements.txt
```